Memory-Centric Scaling of LLM Decoding Through Contextual Sparsity

MCCSys @ ISCA'26

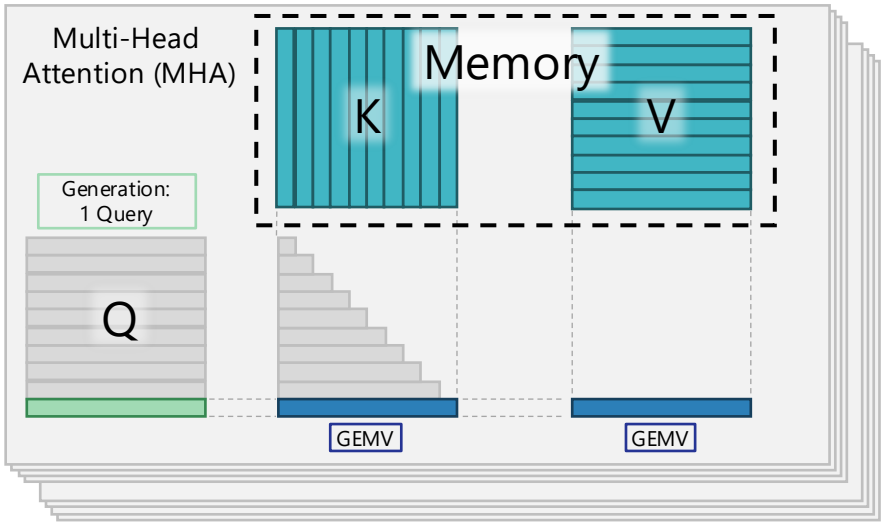
Liu Liu, Assistant Professor

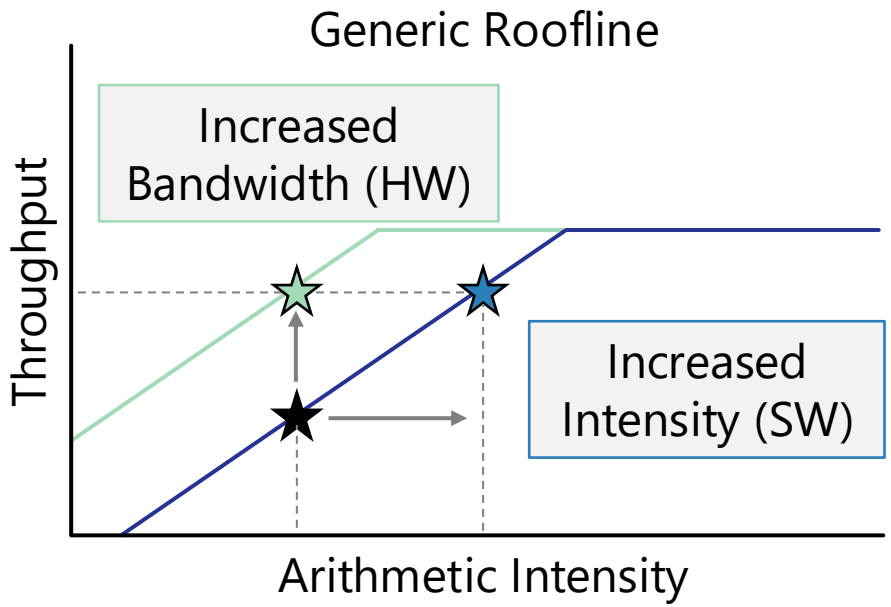
ECSE/CS@RPI

RPI

Let's Talk About Context

- Problem: Large language models (LLM) using self-attention *need increasingly long contexts*
  - GEMV + No reuse → Low arithmetic intensity
- Two avenues for solutions
  - Higher bandwidth → Hardware constrained
  - Higher arithmetic intensity → Algorithmic

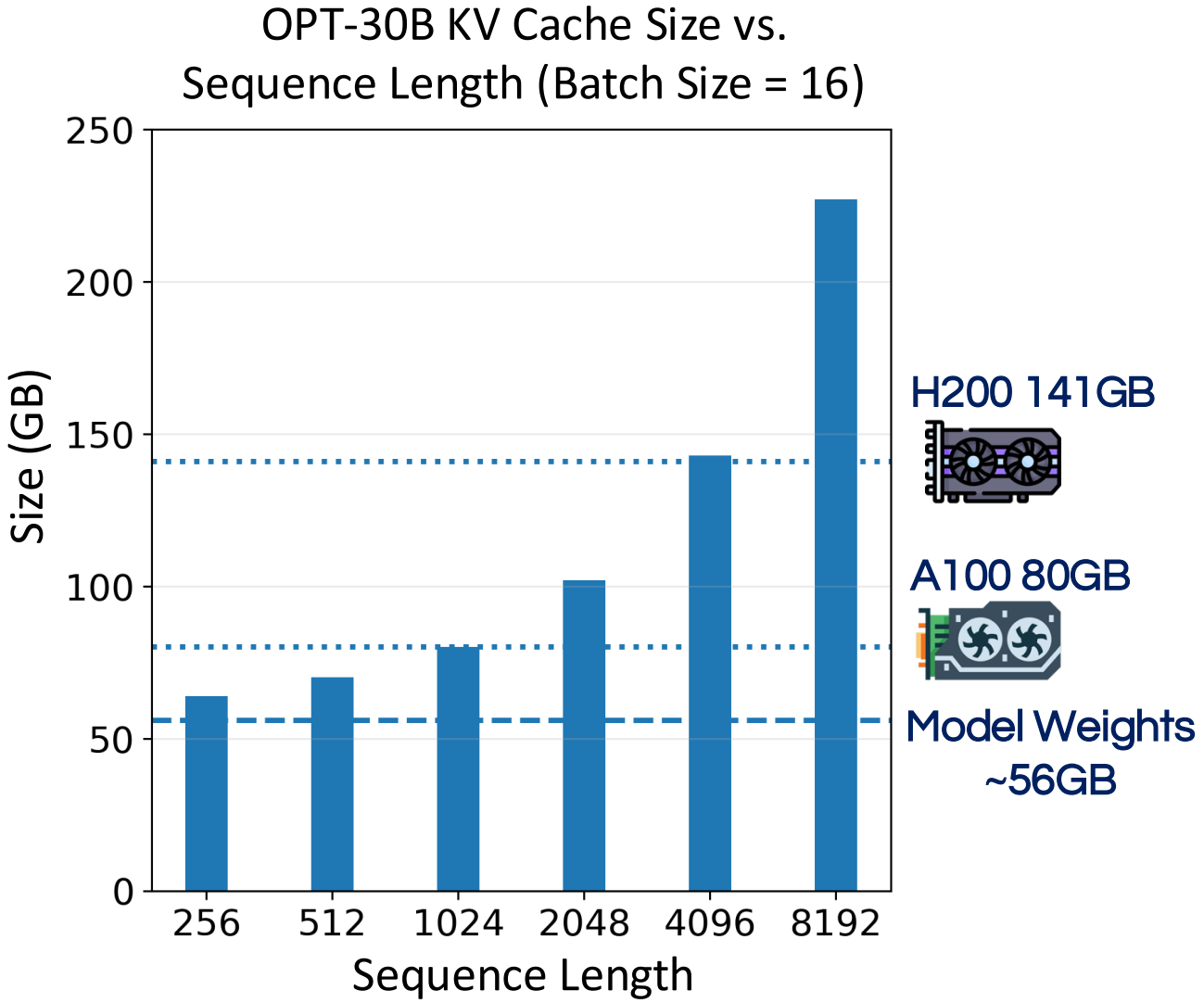




RPI

Growing Context and Memory-Boundness

- LLMs generate outputs in two stages: Prefill and Decoding.
- During decoding, each token attends to **all past key/value vectors.**
- As context length increases, **KV-cache capacity and bandwidth demands grow rapidly.**
- This makes attention increasingly **memory-bound** and dominant in long-context decoding.



RPI

Opportunities with Contextual Sparsity

- **Precision scaling**
  - Reading fewer bits for less-critical values
- Selective token access
  - Not all tokens from context are useful

Contextual Sparsity as Precision Scaling

BitWeaver: Read-Time Truncation in Memory, ICS 2025

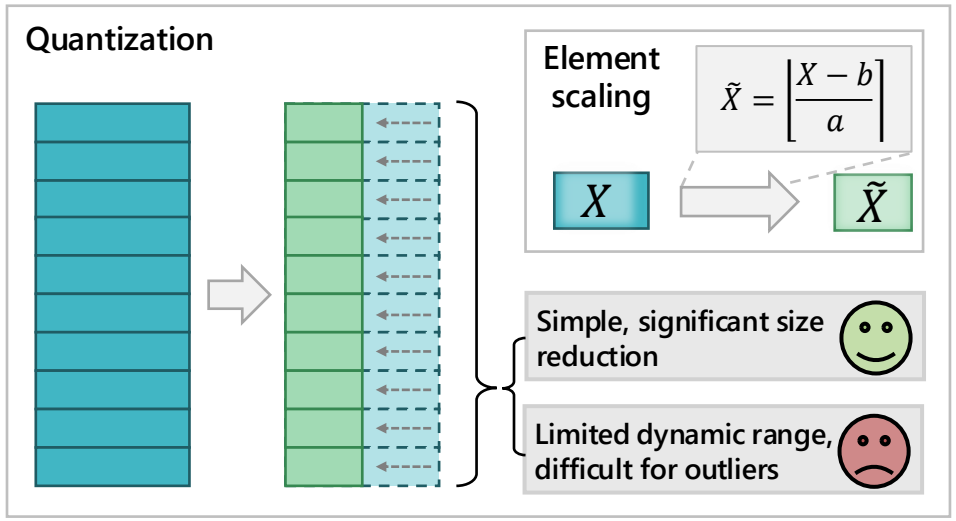
Garrett Gagnon[+], Srikanth Malla*, Yangwook Kang*, Liu Liu[+]

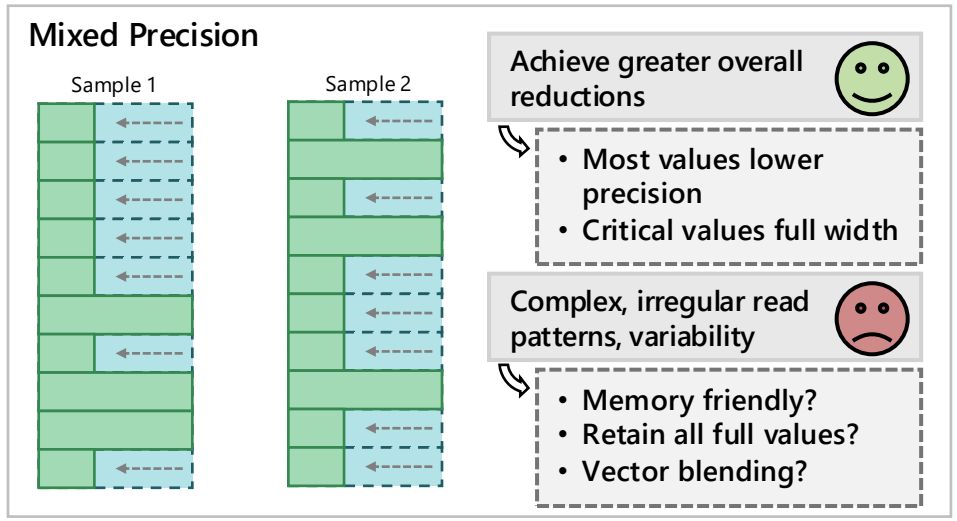
[+]Rensselaer Polytechnic Institute, *Samsung Semiconductor

RPI SAMSUNG

Mixed-Precision Attention

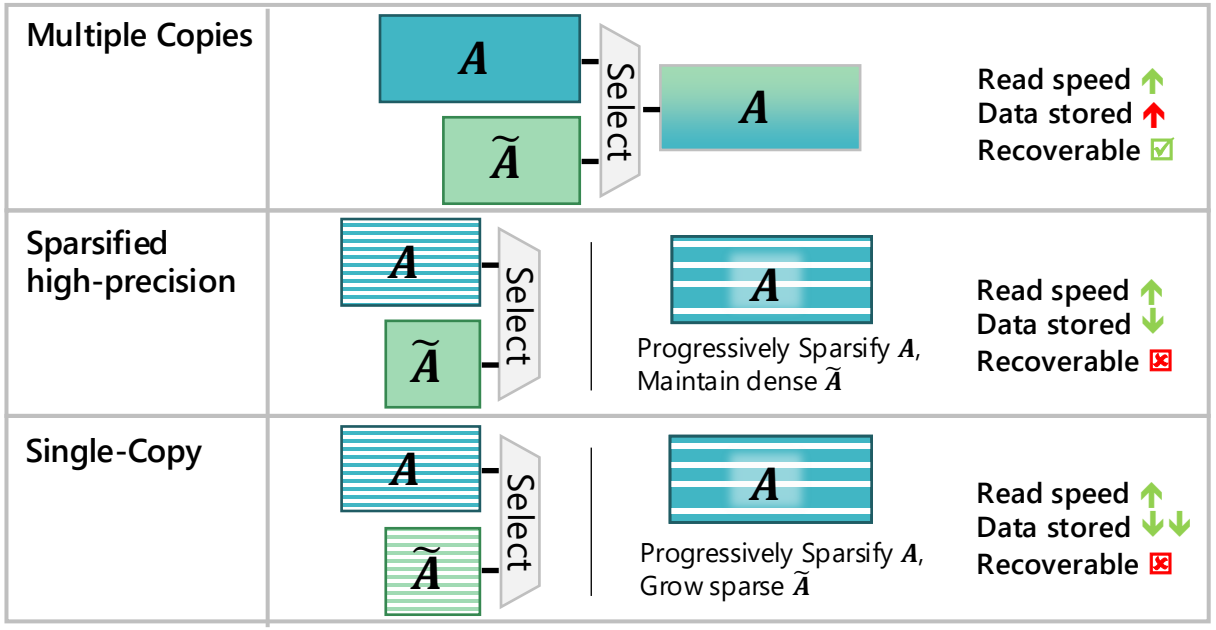
- Potential algorithmic solutions to reduce traffic
  - Quantization
    - Fewer bits → Simple approach, precision-range tradeoff
  - Mixed precision
    - Prioritize critical values → Higher accuracy, more complex

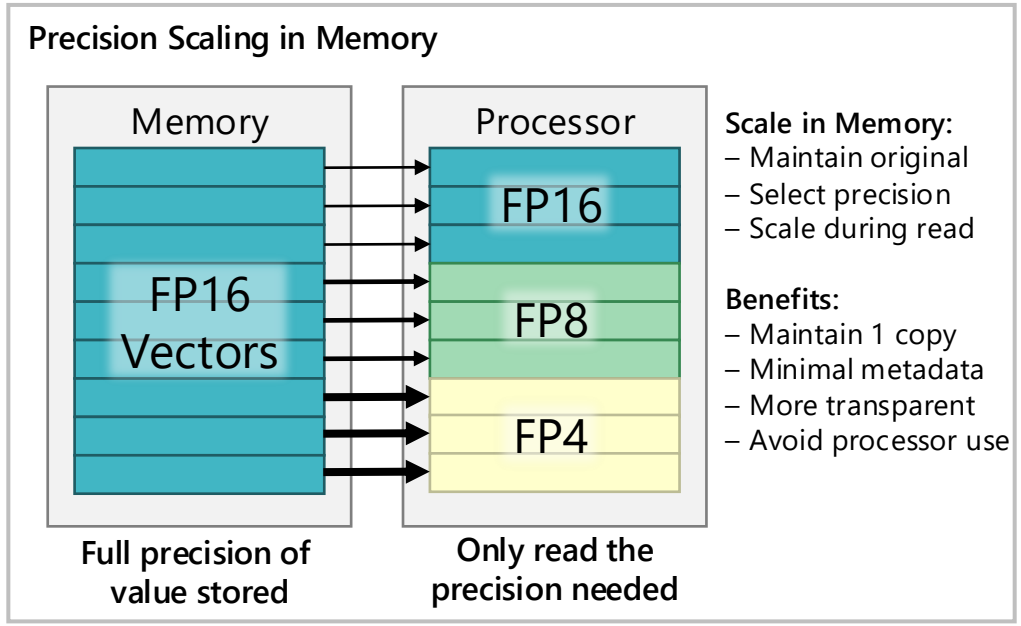




Challenges with Multiple Precisions

- Memory is precision-static
  - Memory can only provide what has already been written
  - Must have the correct precision stored prior to read-time
- Precision-scaling in memory could provide unique benefits



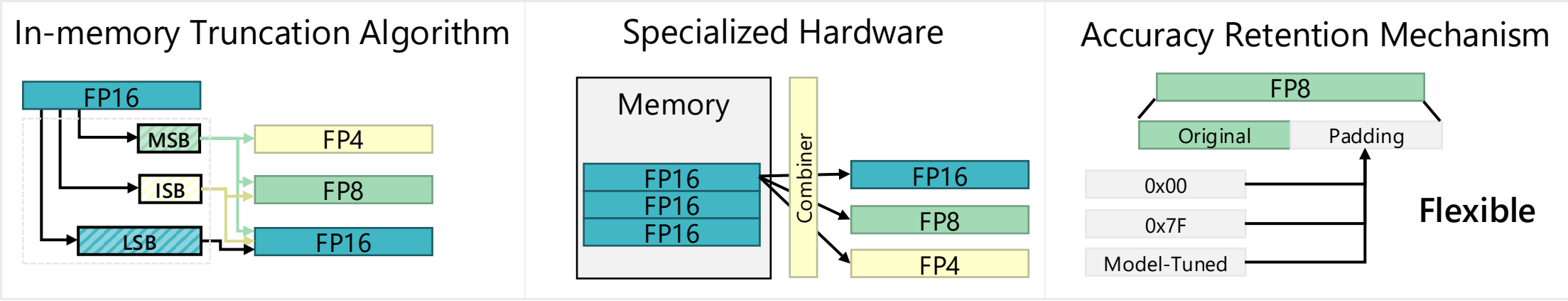


RPI

Dynamic Precision Scaling in Memory

To accelerate LLM self-attention, we present:

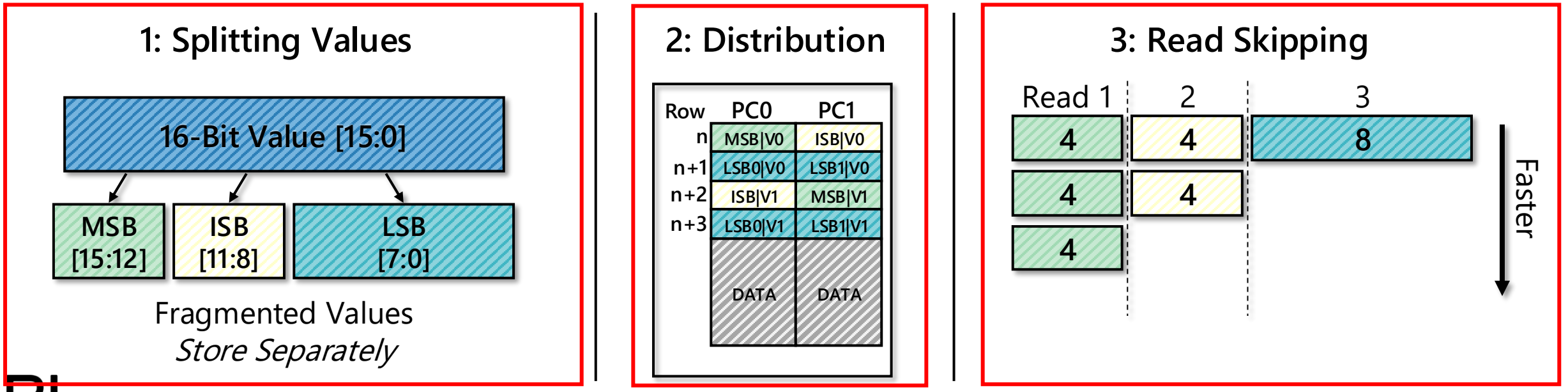
1. A hardware-aware in-memory truncation algorithm.
2. A specialized hardware implementation.
3. An accuracy retention mechanism.
4. A detailed analysis of the trade-offs between memory traffic reduction and accuracy preservation.



RPI

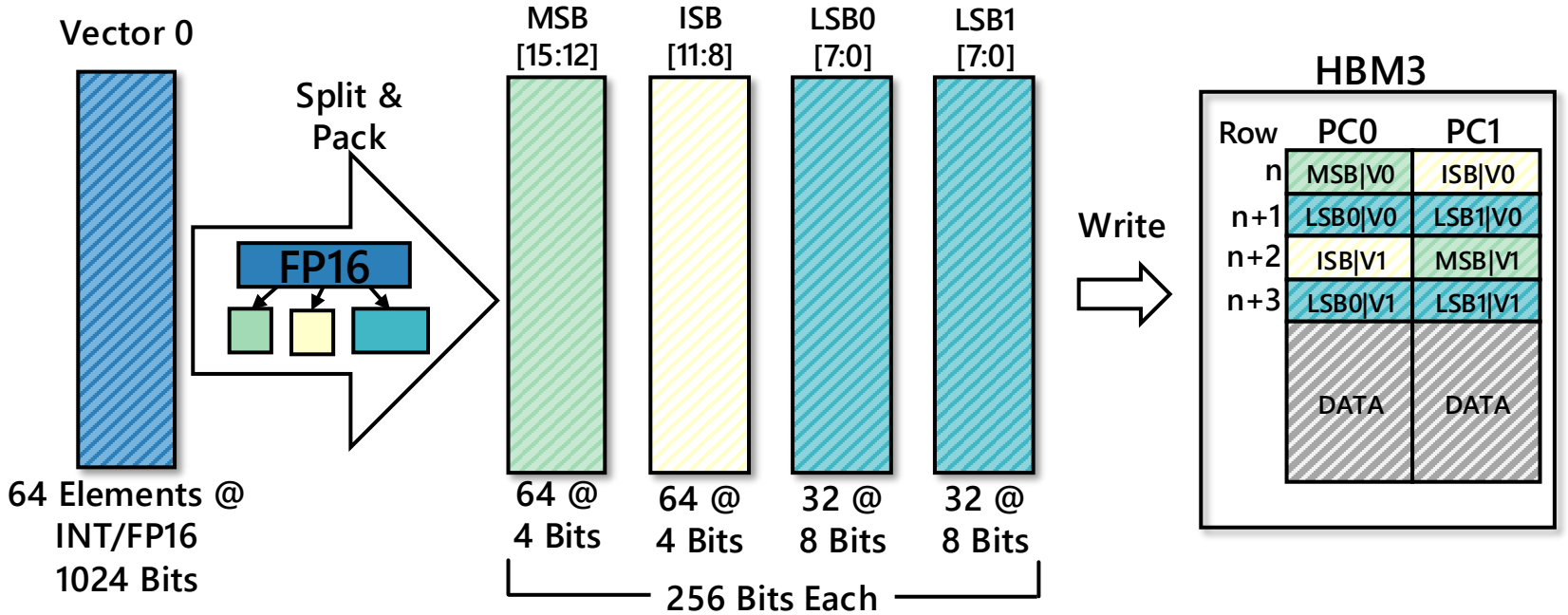
Effective Truncation in Memory

- Limit: We cannot modify memory architecture
- Solution: We modify how data is stored to enable truncation
  1. Split values
  2. Distribute across multiple reads
  3. Skipping reads → lower precision, less time



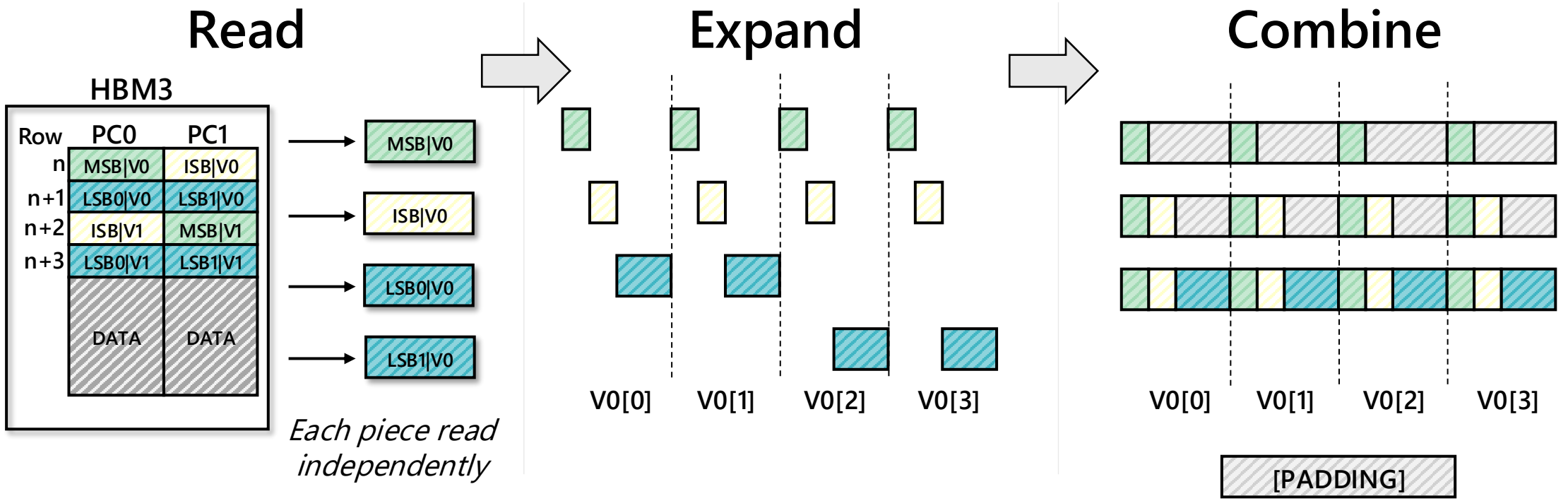
Memory-aware Data Fragmentation

- To account for memory granularity, must vectorize
  - HBM3 Pseudo-Channel: 32 Bits/PC, 8 Reads/Burst → 256 Bit chunks
  - Granularity from smallest chunk: 4 Bits → Pack 64 Elements



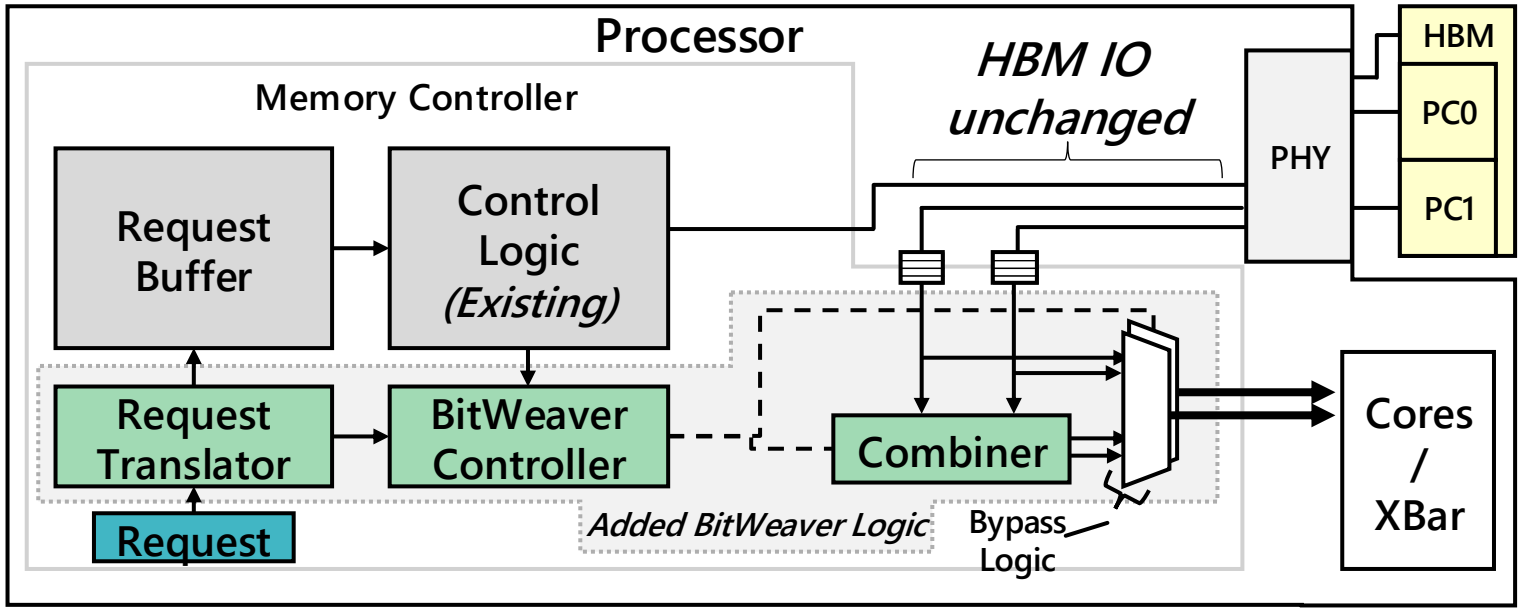
Reconstruction Process

- Reconstruction is kept simple
  - No arithmetic
  - Fragments concatenated together



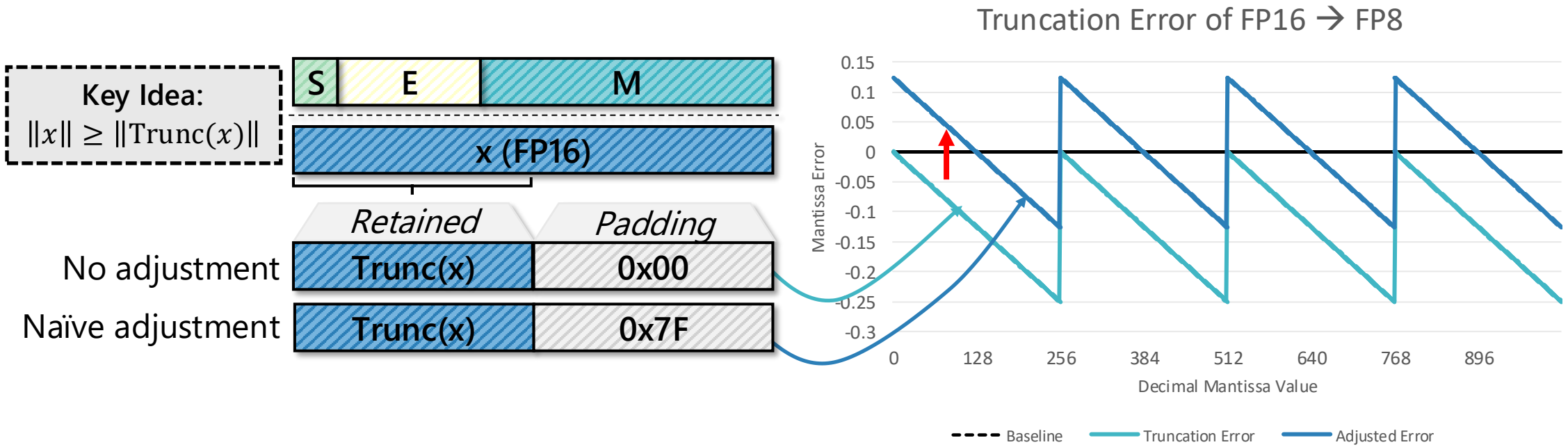
Hardware Integration

- For HBM: Integrate with controller
  - On processor → high bandwidth post-expansion
    - No changes to HBM I/O



Mitigating Truncation Error

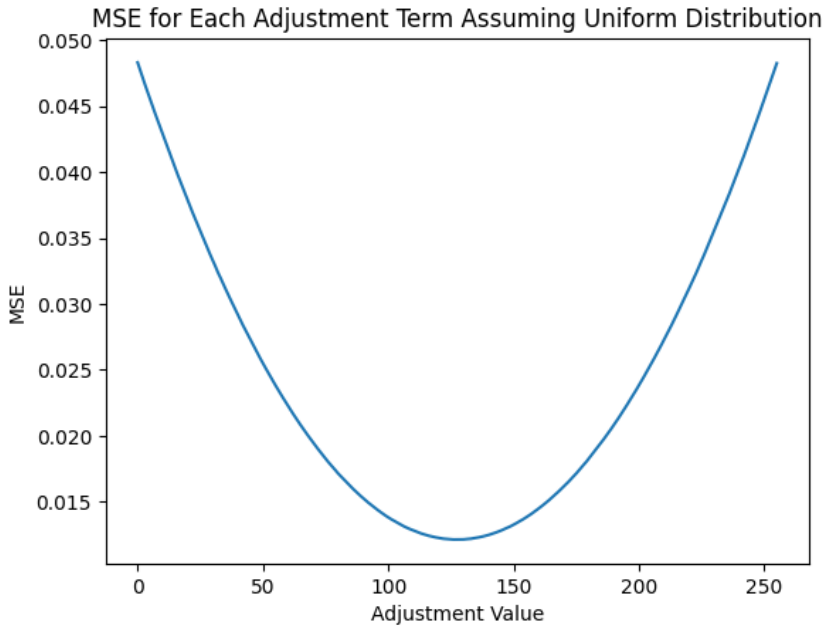
- Truncating values adds error
- Observation: Truncation is always reductive to magnitude
  - Solution: Pad with nonzero values to reduce the mean error
  - A naïve choice for 8-bit value might be to use 0x7F

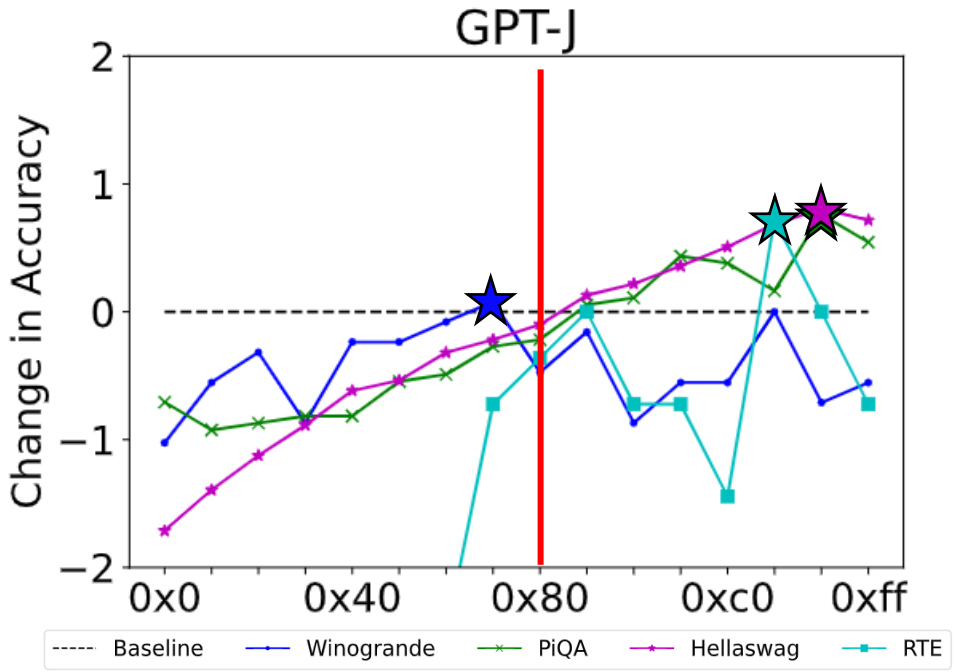


$$\mathrm{Trunc}(x) = x \,\&\, 0\mathrm{xFF00}$$

Sensitivity of Adjustment Value

- Per-element vs. model error minimization
  - Testing shows 0x7F to be good, but not the best
  - Influence of over- & under- estimations
  - Potential to identify a model's best value

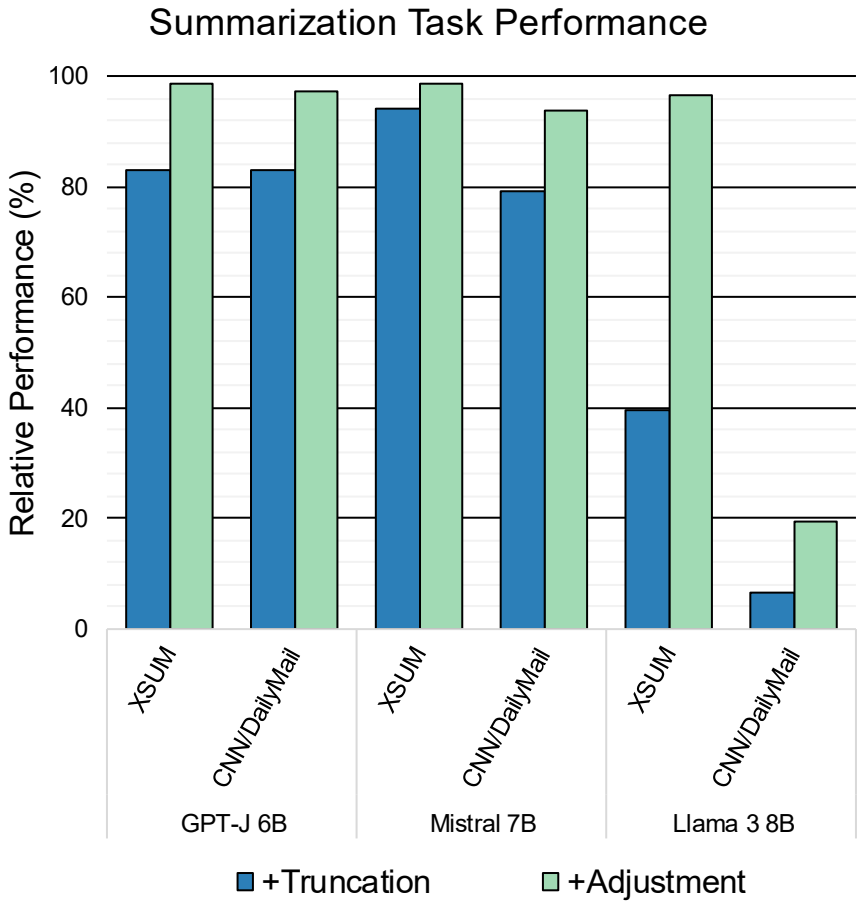




RPI

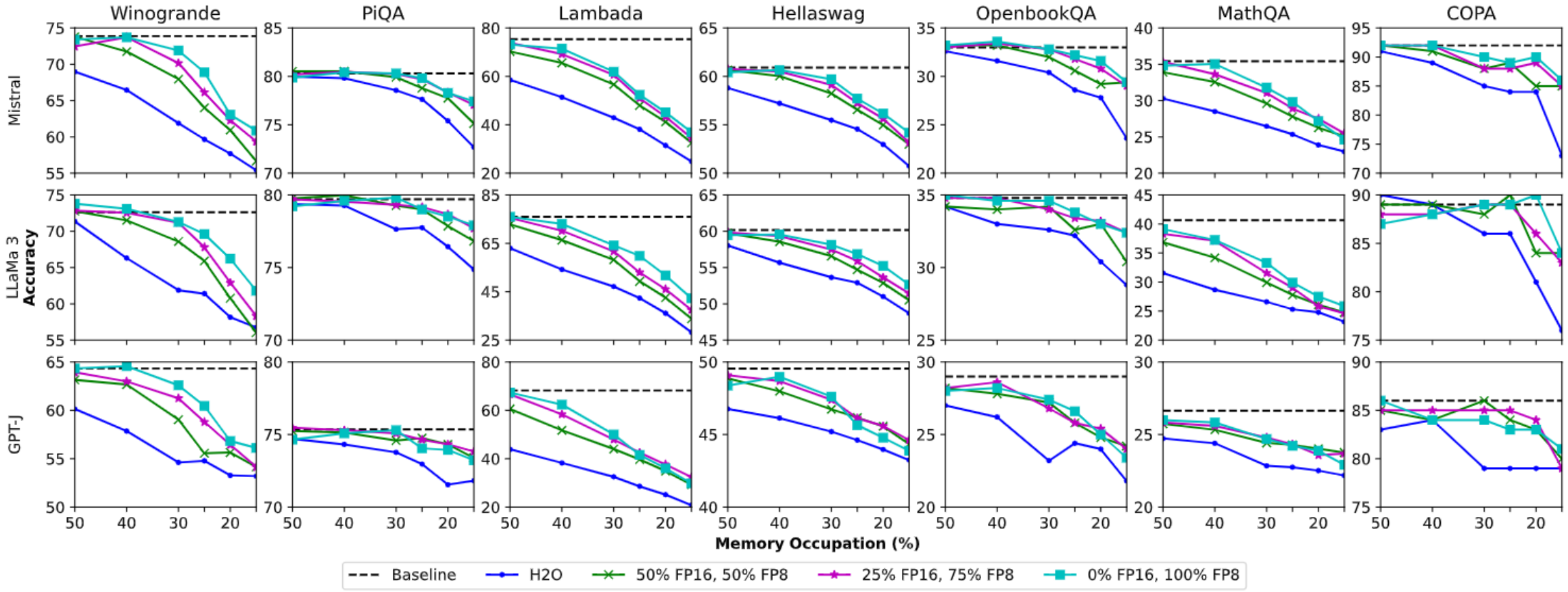
Adjustment Recovers Truncation Error

- Simple truncation can cause performance degradation
- Per-task searched adjustment improves performance
- Uniformly applied FP8 truncation



RPI

Access More Tokens Approximately

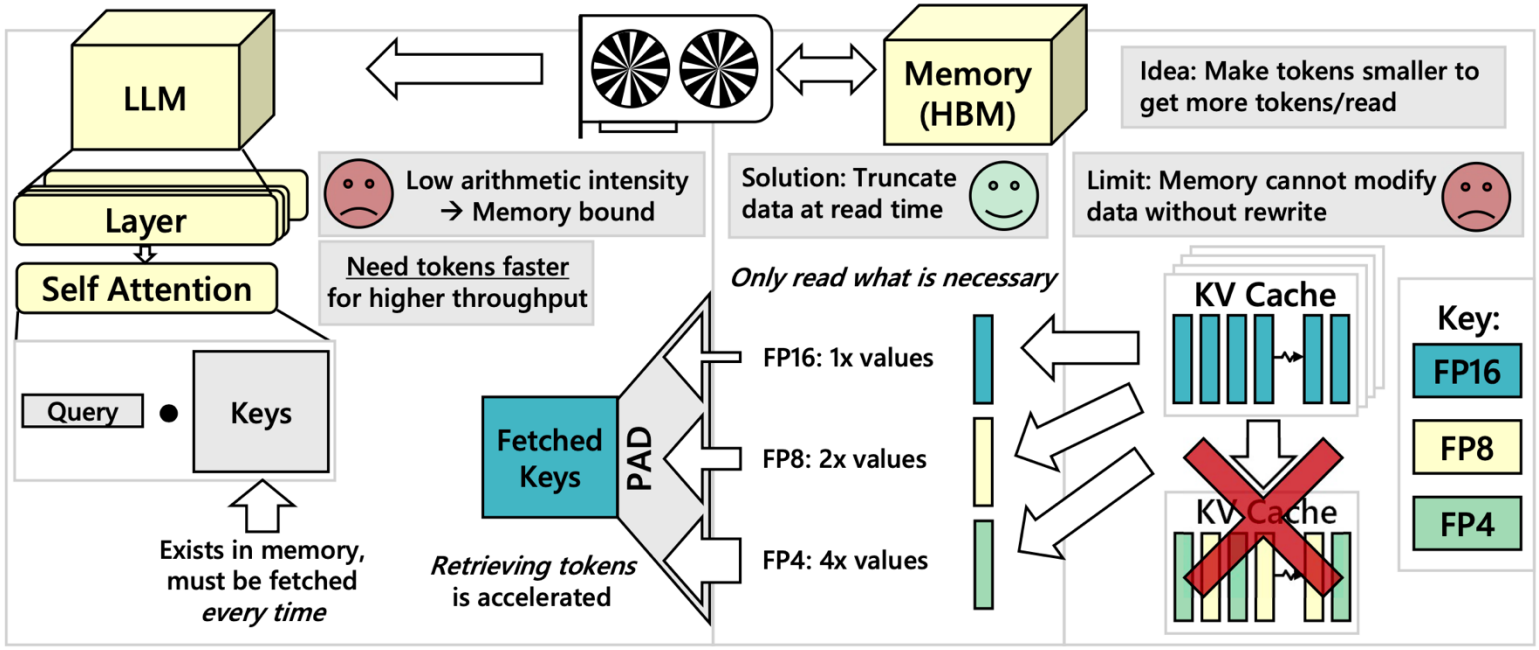


Read more tokens @ FP8 or mixed vs. all in FP16 under the same memory budget

RPI

Contextual Sparsity as Precision Scaling

- Memory bottleneck of accessing contextual data
- In-memory truncation: read what you need from memory
  - 3x effective memory bandwidth
  - 1.8x speedup



Opportunities with Contextual Sparsity

- Precision scaling
  - Reading fewer bits for less-critical values
- **Selective token access**
  - Not all tokens from context are useful

Contextual Sparsity as Selective Token Access

STARC: Selective Token Access with Remapping and Clustering for Efficient LLM Decoding on PIM Systems, ASPLOS 2026

Zehao Fan[1], Yunzhen Liu[2], Garrett Gagnon[1], Zhenyu Liu[1], Yayue Hou[1], Hadjer Benmeziane[3], Kaoutar El Maghraoui[4], Liu Liu[1]

[1]Rensselaer Polytechnic Institute, [2]University of Massachusetts, [3]IBM Research – Ruschlikon, [4]IBM T. J. Watson Research Center

RPI IBM

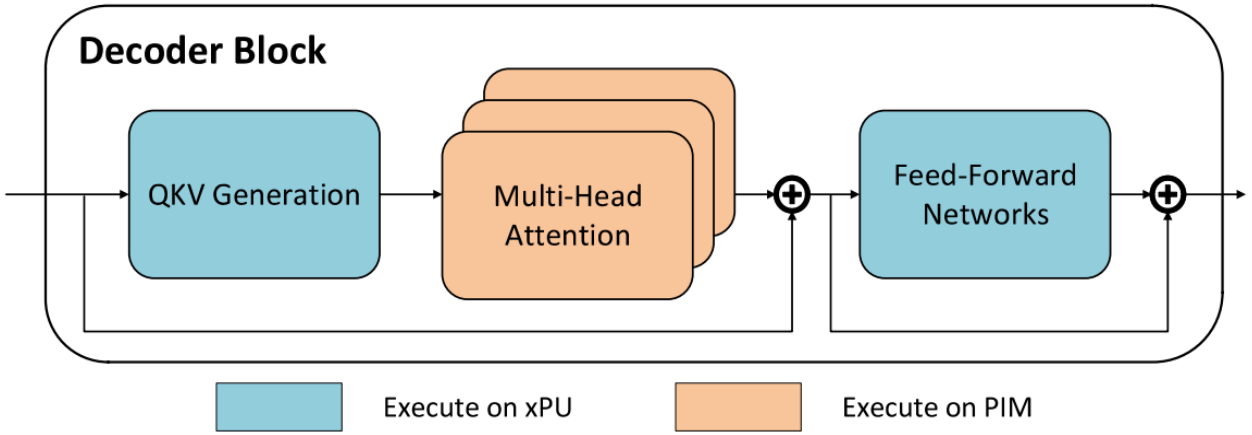
Why PIM for Long-Context LLM Decoding

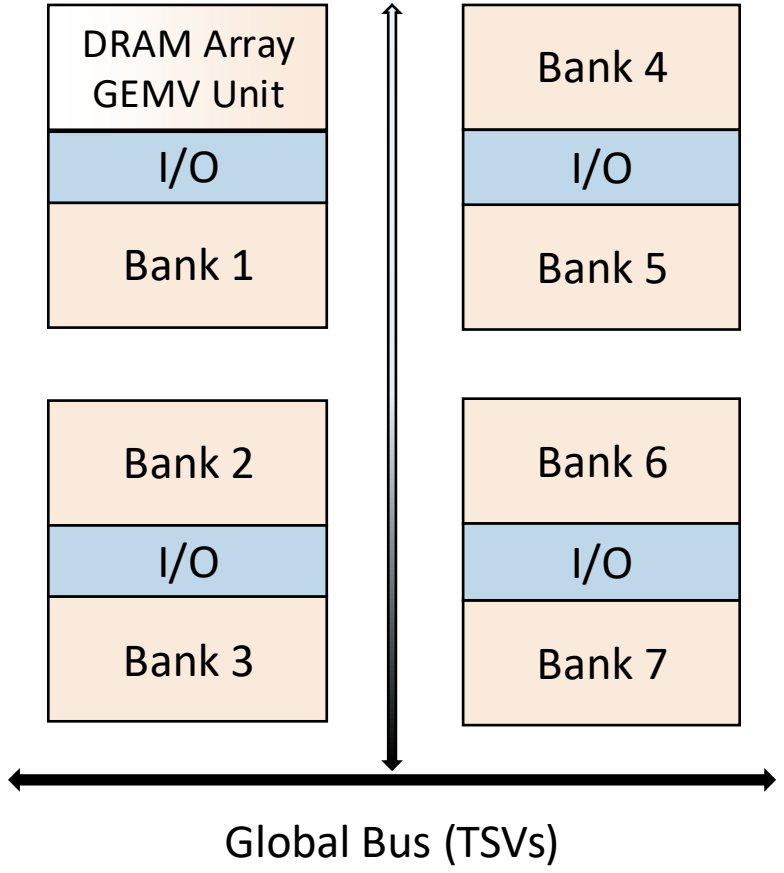
- Processing-in-memory (PIM) architecture places simple compute units near or inside memory.

High Internal Bandwidth

Bank-Parallel Operations

- These make PIM a good match for **memory-bound attention.**

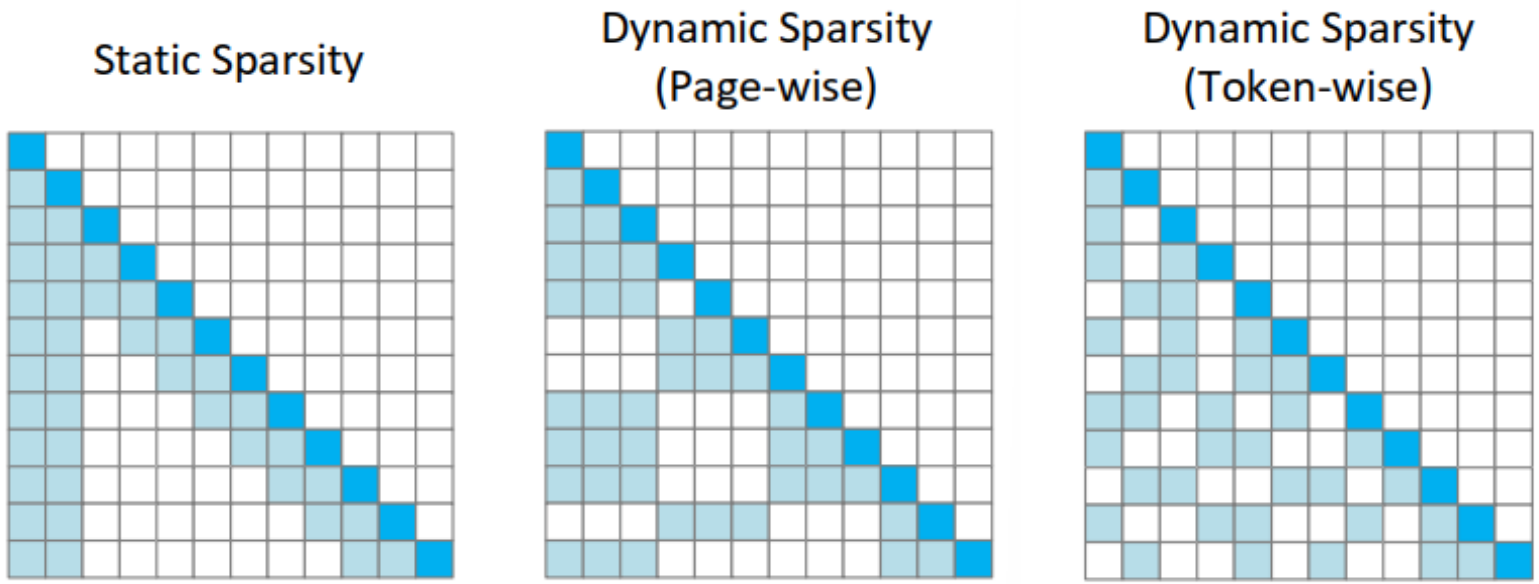




RPI

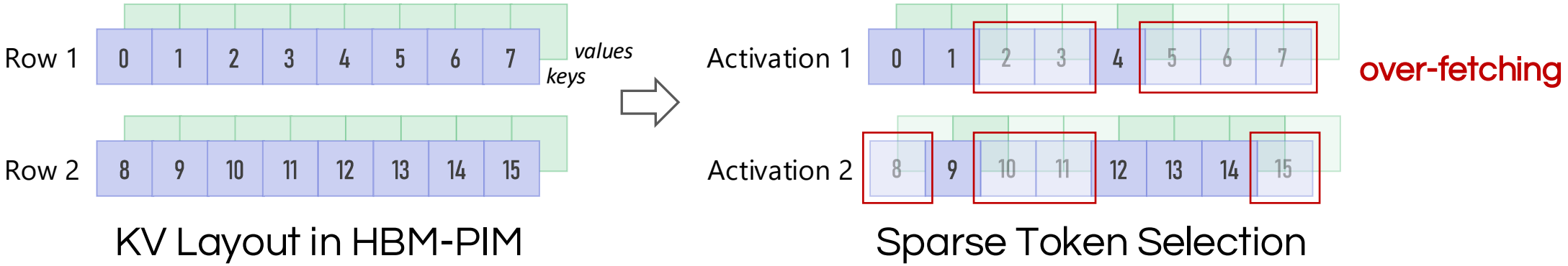
Sparse Attention: Granularity Matters

- KV Cache Sparsity: A small portion of critical tokens will dominate the attention outcomes.



Mismatch between **Semantic Sparsity** and **Row-level Memory Access**

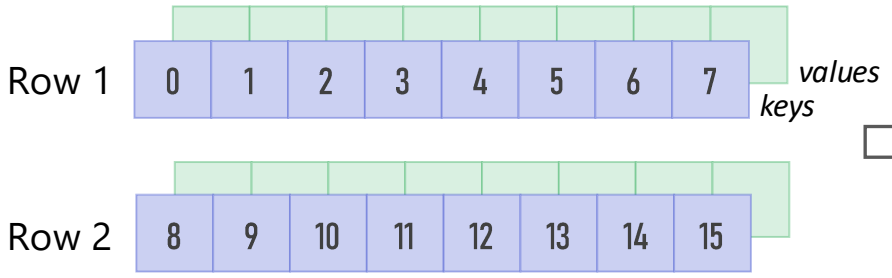
Problem: Sparse decoding is promising, but hard to execute efficiently on **PIM.**



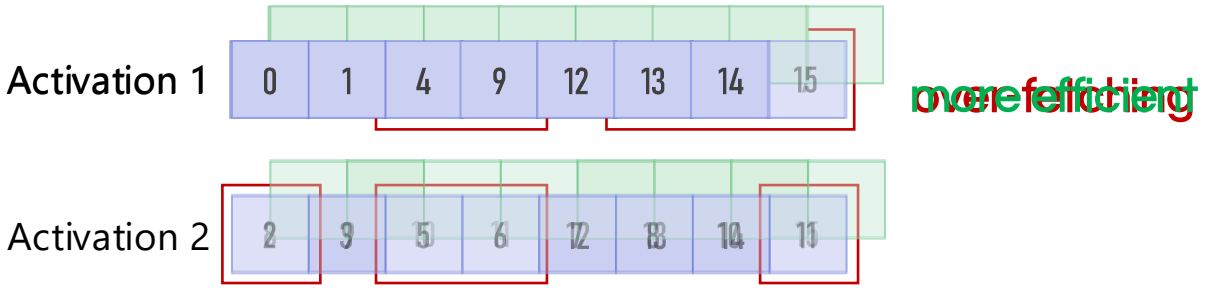
RPI

Exploit Semantic Similarity

Our Idea: **STARC** clusters semantically similar KV pairs and remaps them contiguously. This turns sparse retrieval into **PIM-friendly row-level access.**



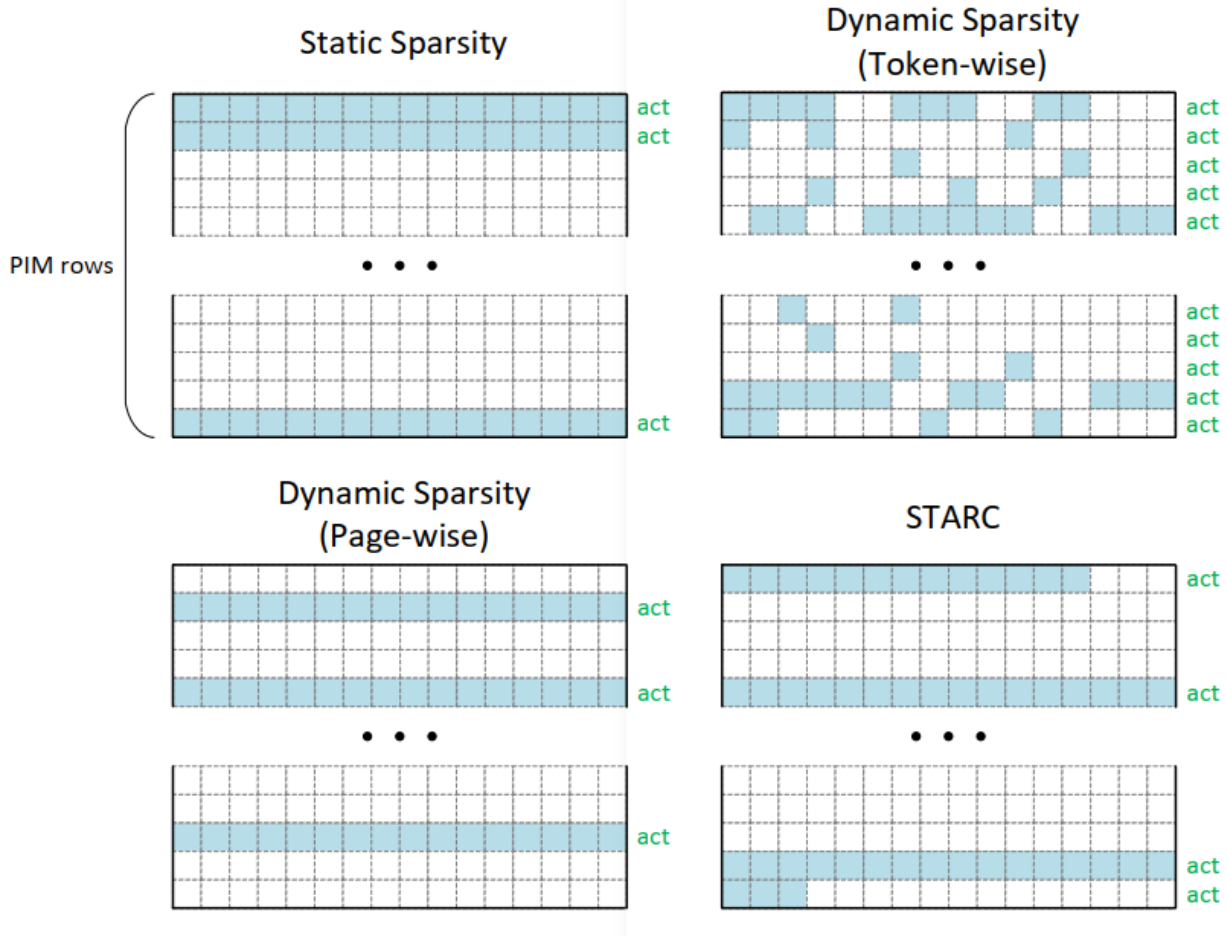
KV Layout in HBM-PIM



Sparse Token Selection

RPI

Token-Wise Sparsity is Inefficient on PIM

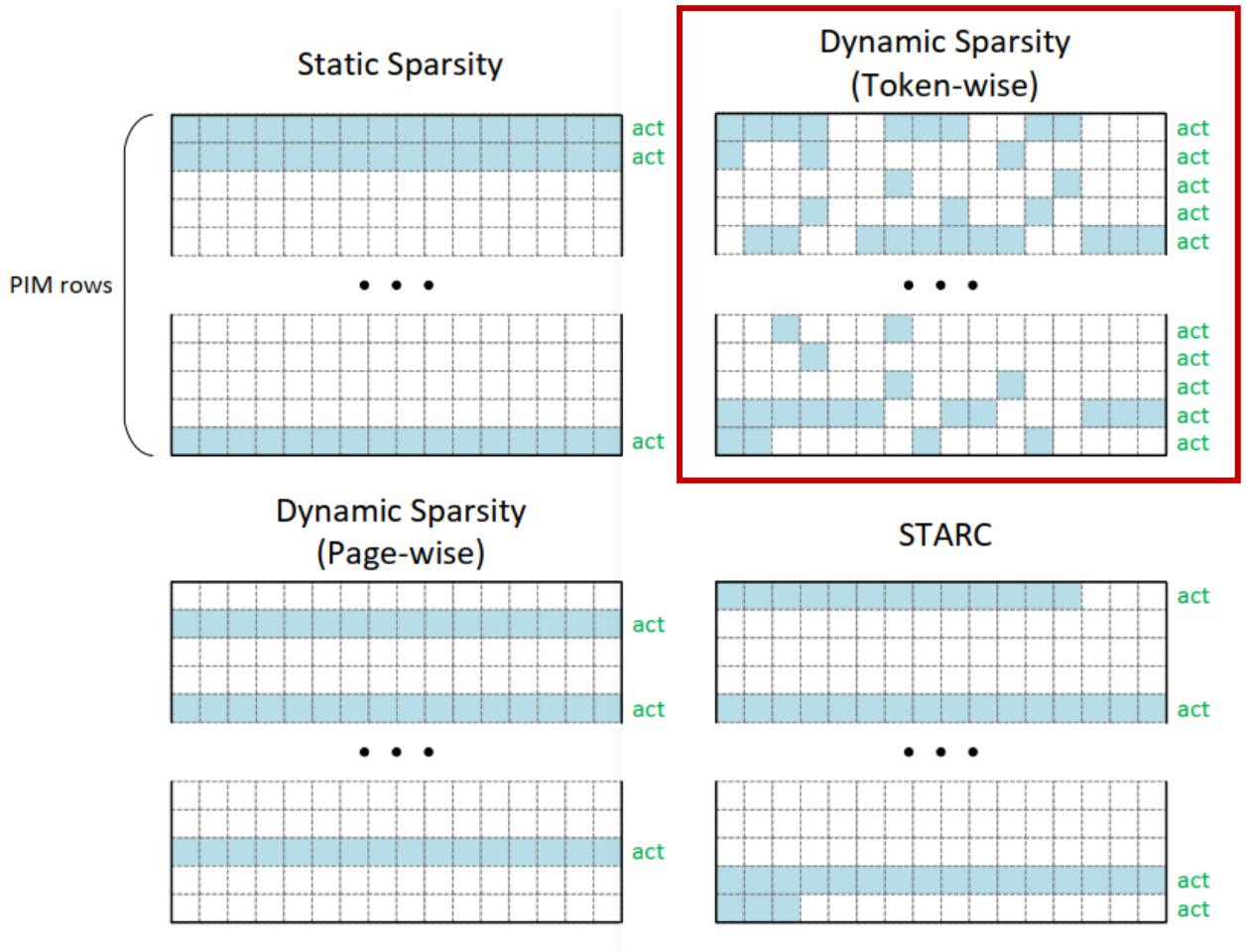


RPI

Token-Wise Sparsity is Inefficient on PIM

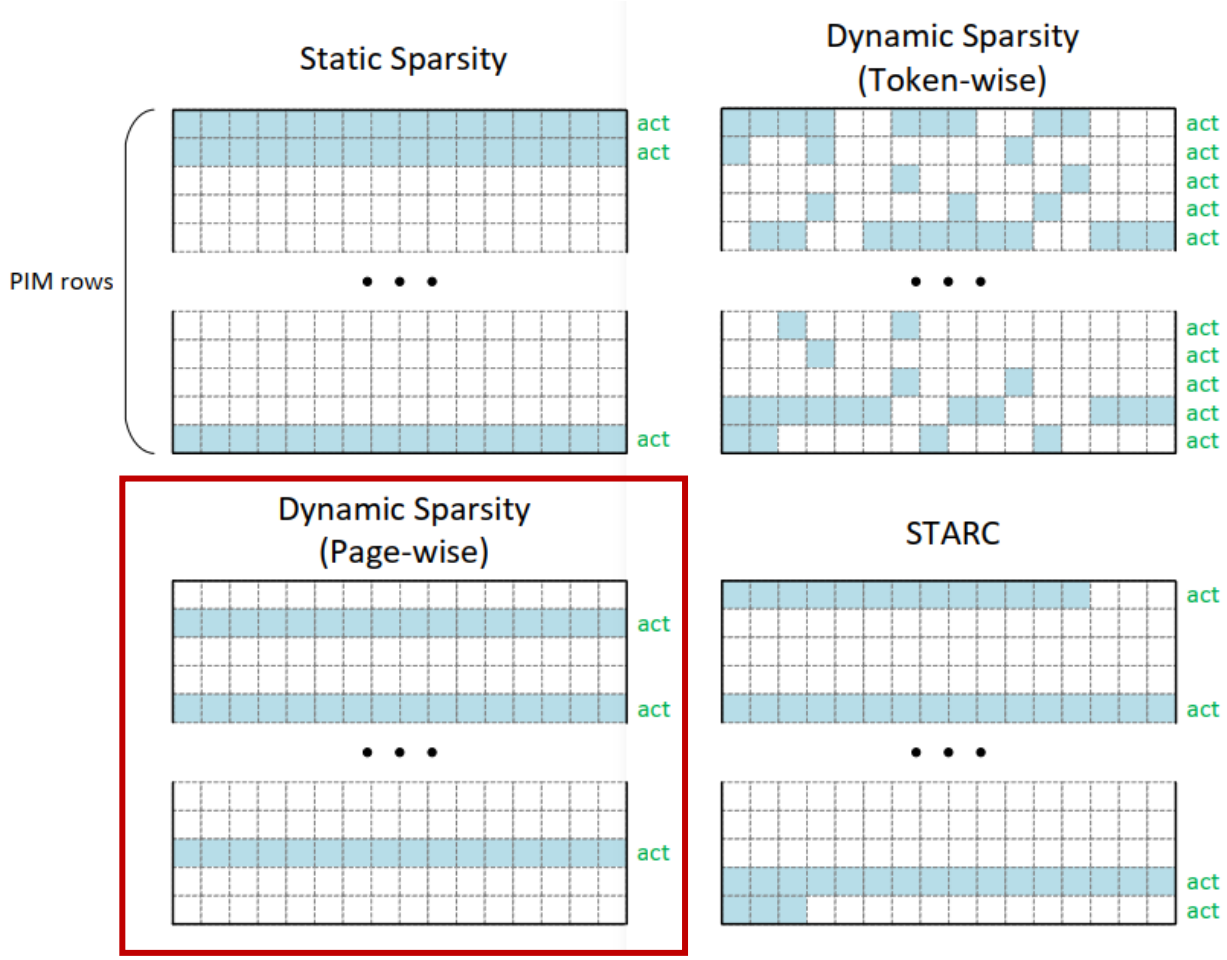
- PIM executes at **row granularity.**
- If relevant tokens are scattered, each row still needs activation.

Token-wise sparsity often leads to **over-fetching, poor bank utilization** and **redundant computation.**



Page-Wise Sparsity Loses Semantic Relevance

- Page-wise retrieval matches row layout.

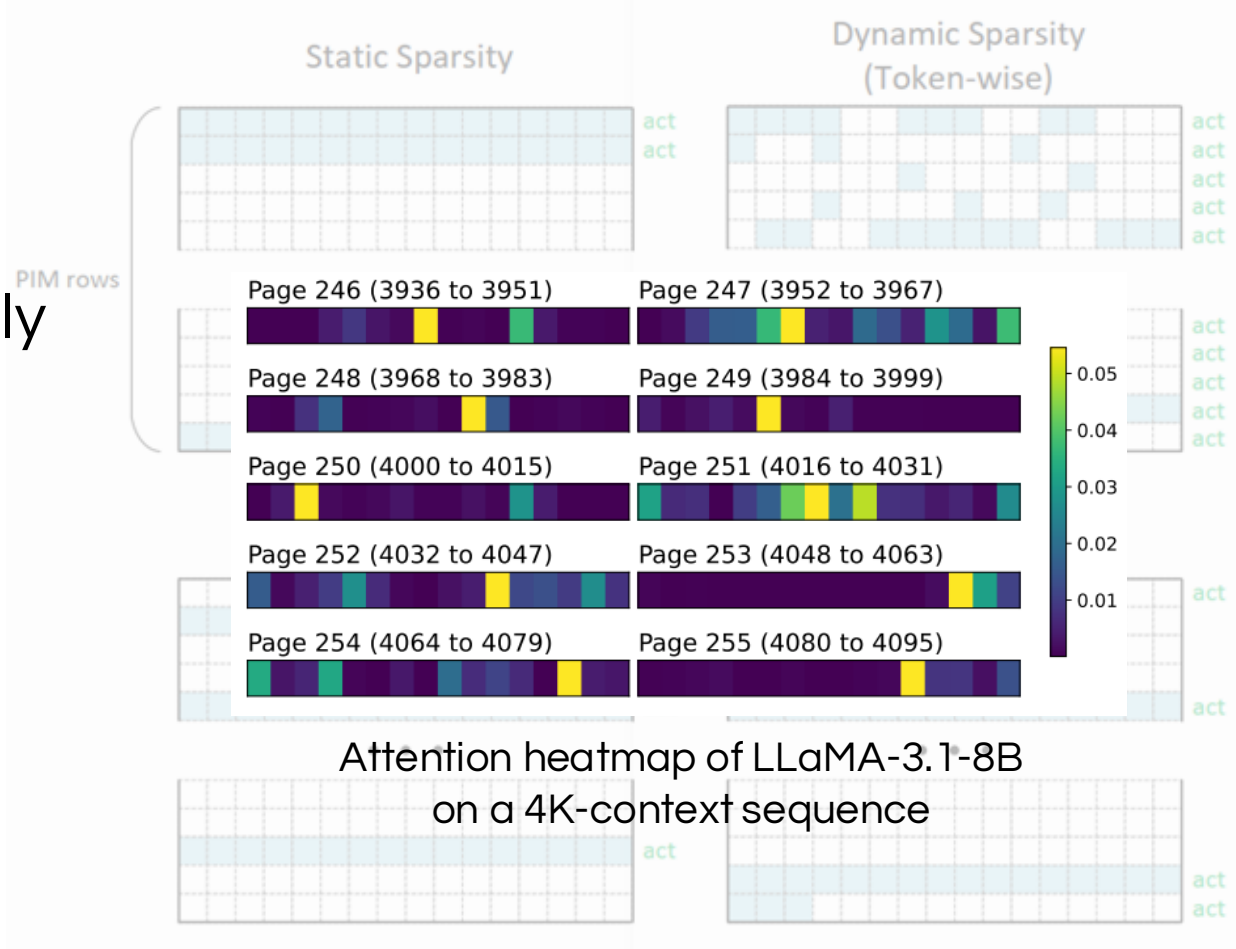


RPI

Page-Wise Sparsity Loses Semantic Relevance

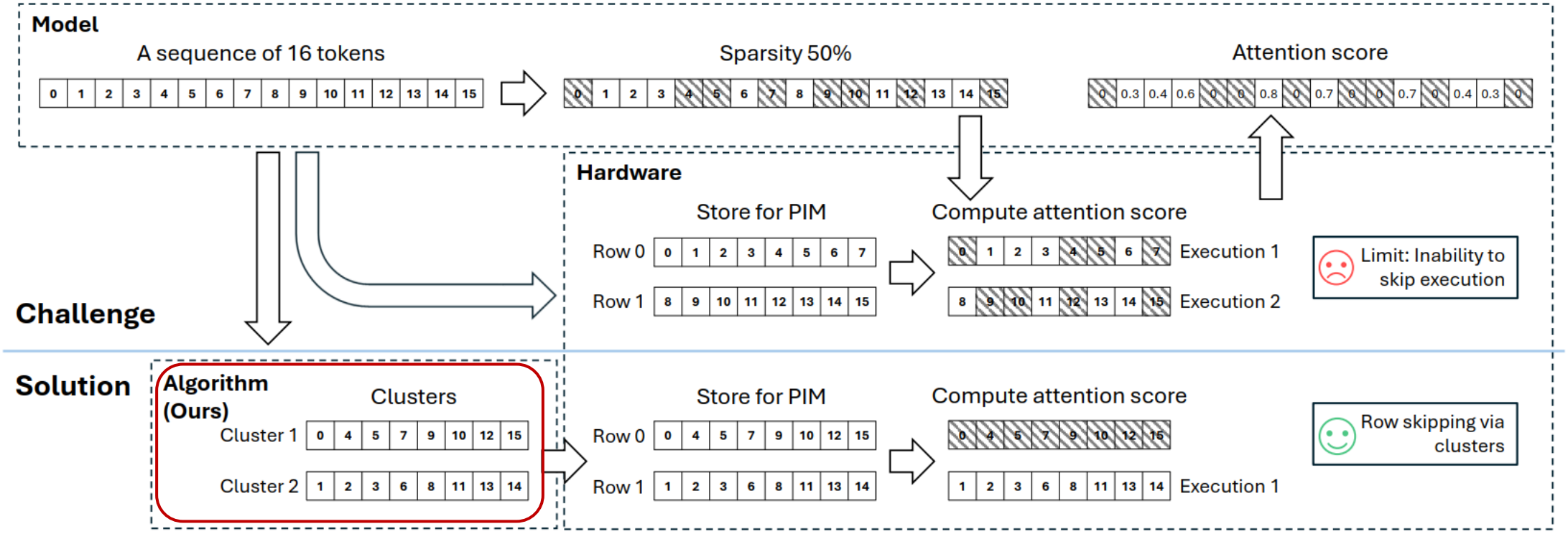
- Page-wise retrieval matches row layout.
- But many selected pages contain only 1–2 important tokens.

Page-wise sparsity's coarse-grained selection leads to a **degradation in model accuracy.**



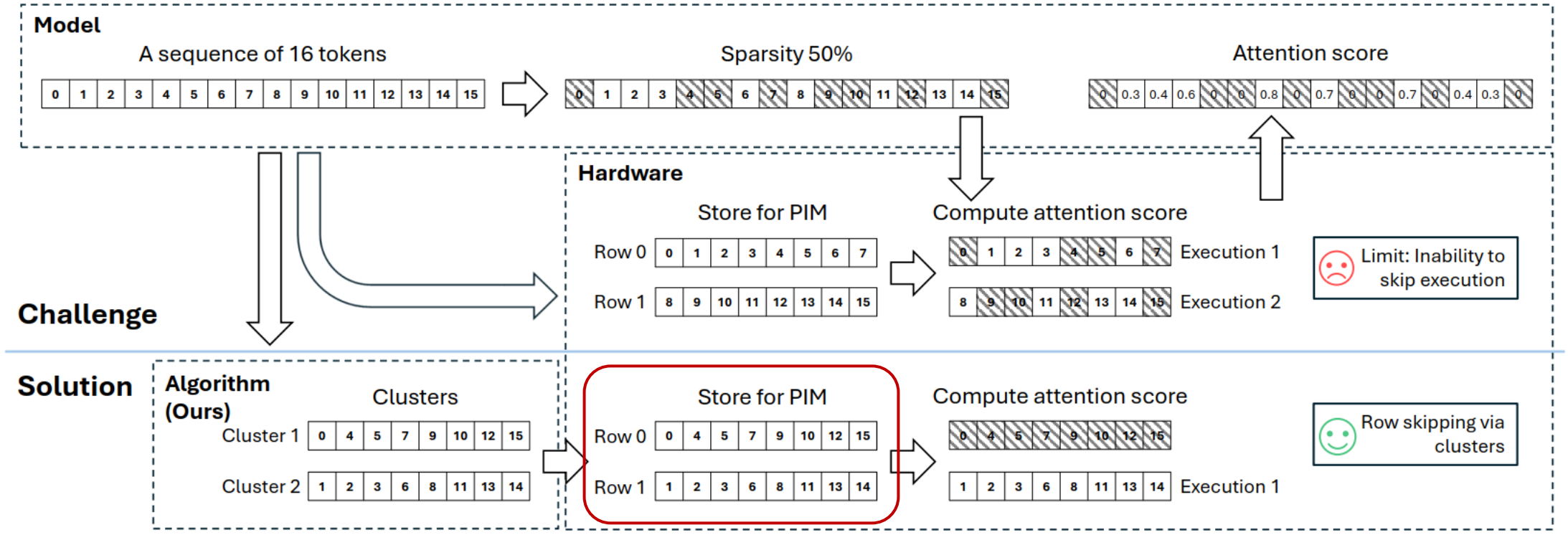
Attention heatmap of LLaMA-3.1-8B on a 4K-context sequence

STARC: Remap Semantic Locality into Row Locality



- Cluster similar KV pairs

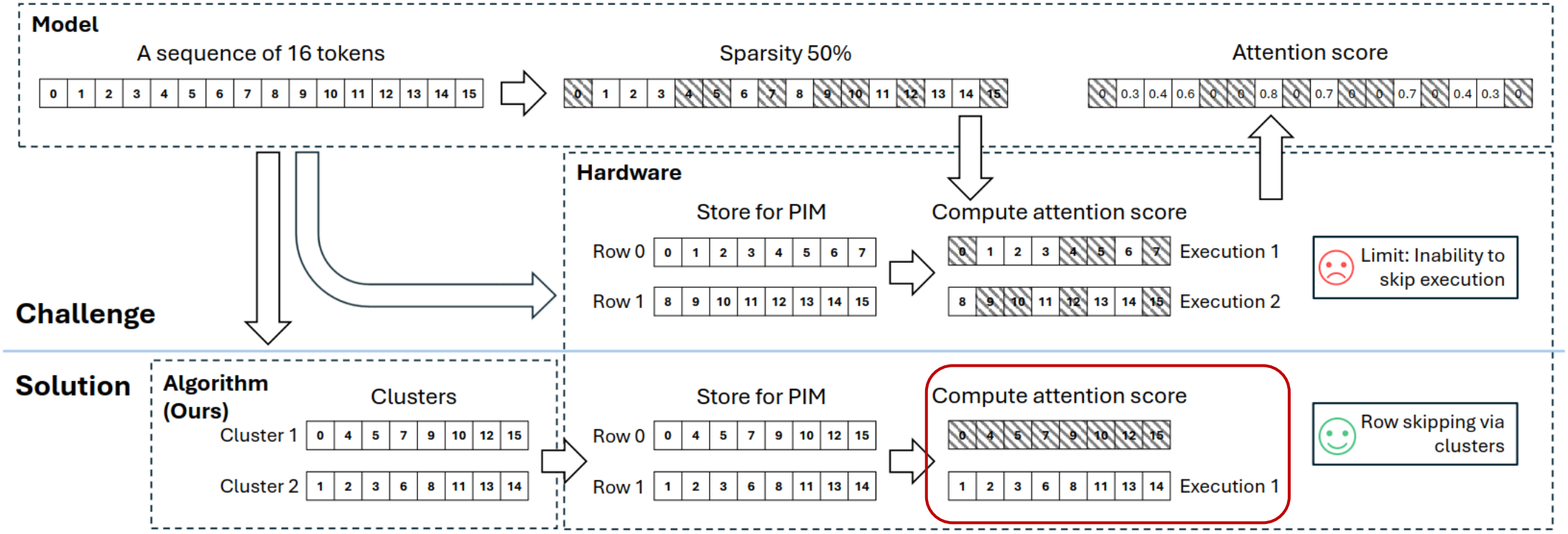
STARC: Remap Semantic Locality into Row Locality



- Cluster similar KV pairs
- Map each cluster to contiguous PIM rows

RPI

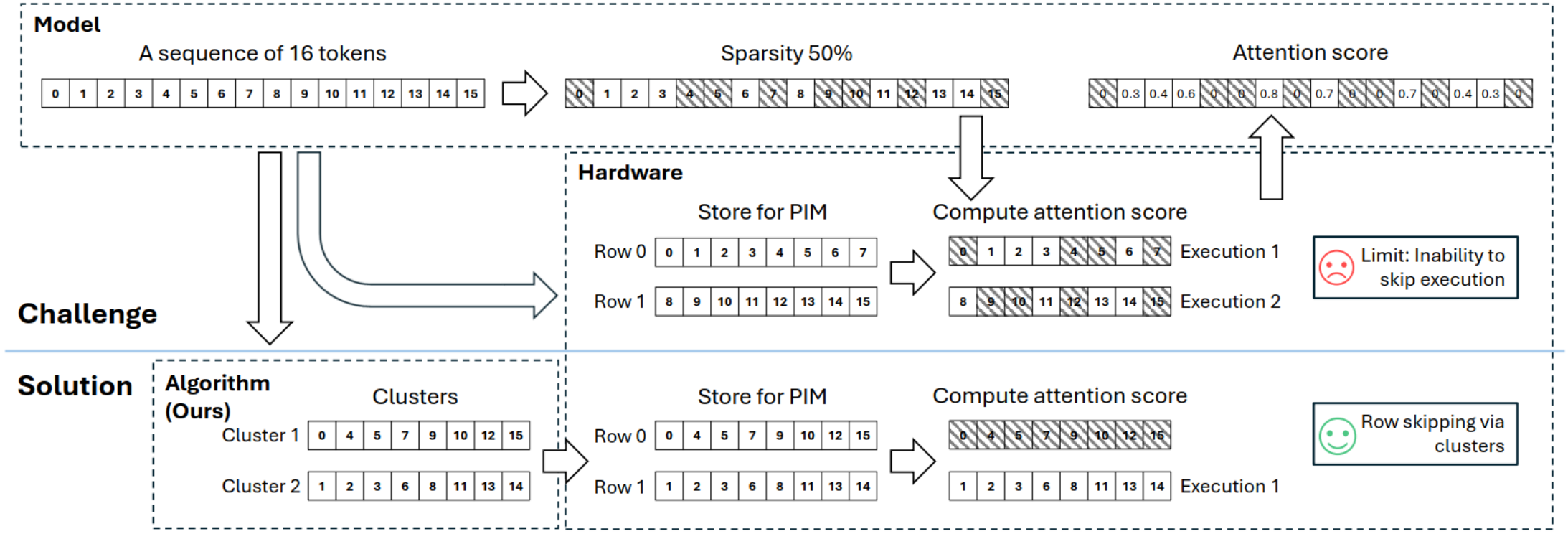
STARC: Remap Semantic Locality into Row Locality



- Cluster similar KV pairs
- Map each cluster to contiguous PIM rows
- Retrieve clusters instead of individual tokens

RPI

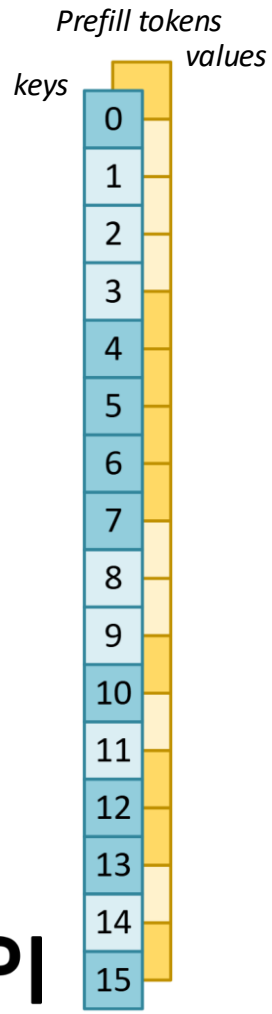
STARC: Remap Semantic Locality into Row Locality



- STARC enables higher important token density within each activated row.
- PIM-side clustering on KV data

Incremental Semantic Clustering

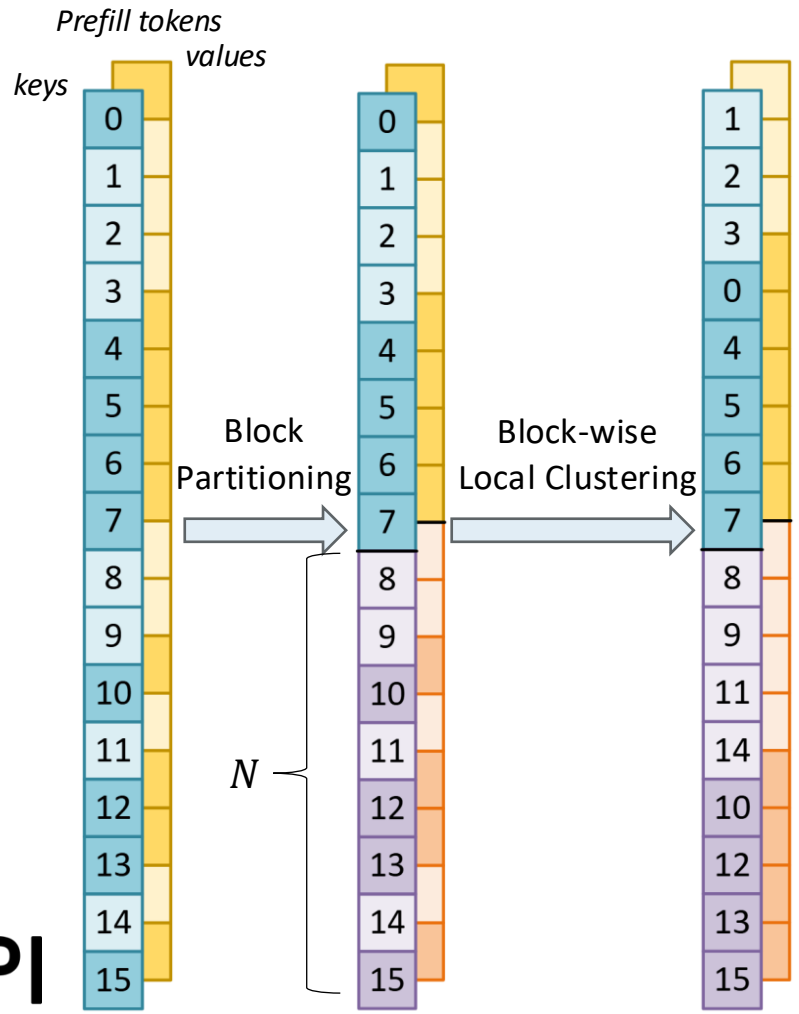
- Set N = Access Granularity × K = 16 × 4 = 64



RPI

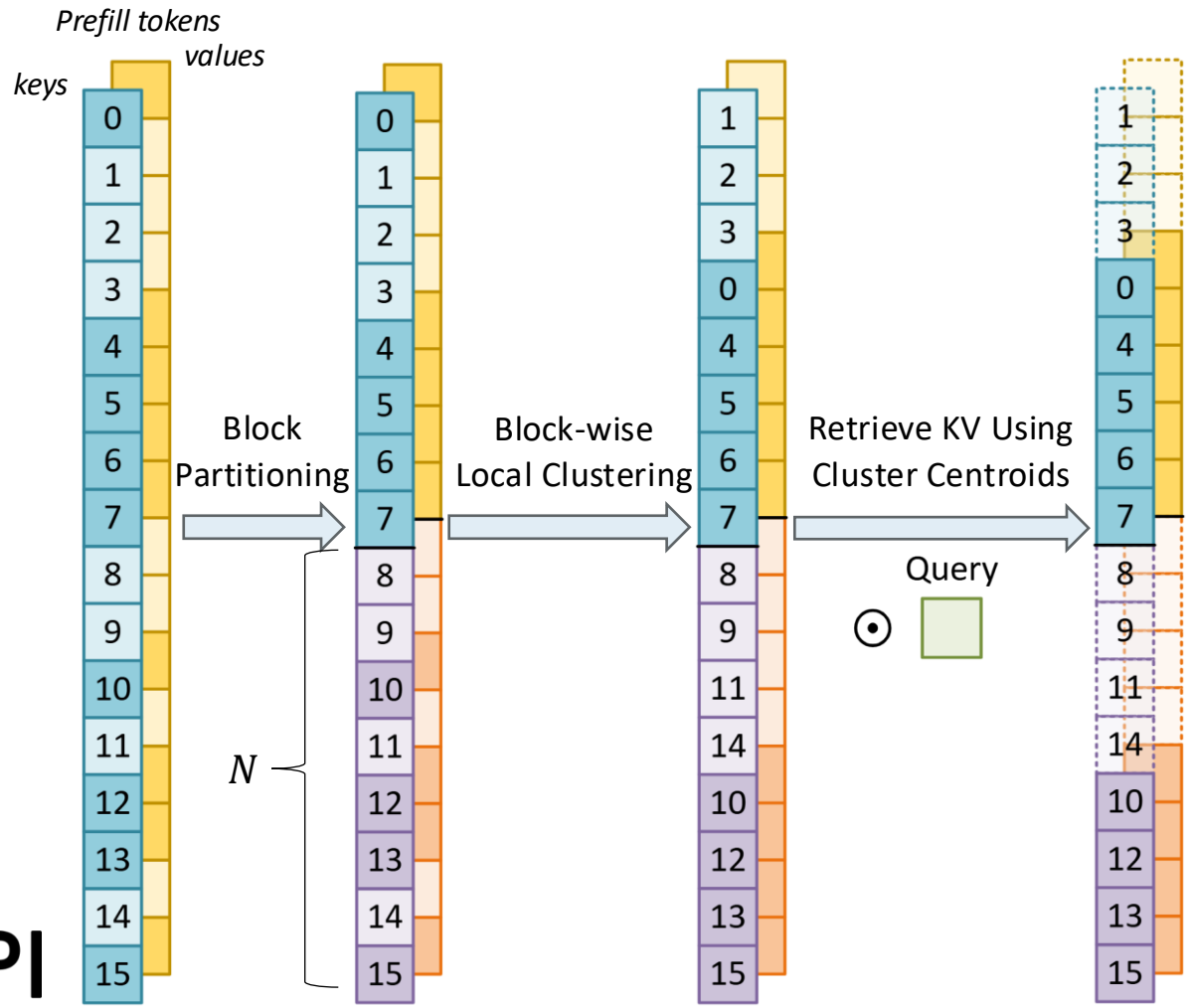
Incremental Semantic Clustering

- **After Prefill:** block partition (blocksize N) → cluster each block → store clusters contiguously



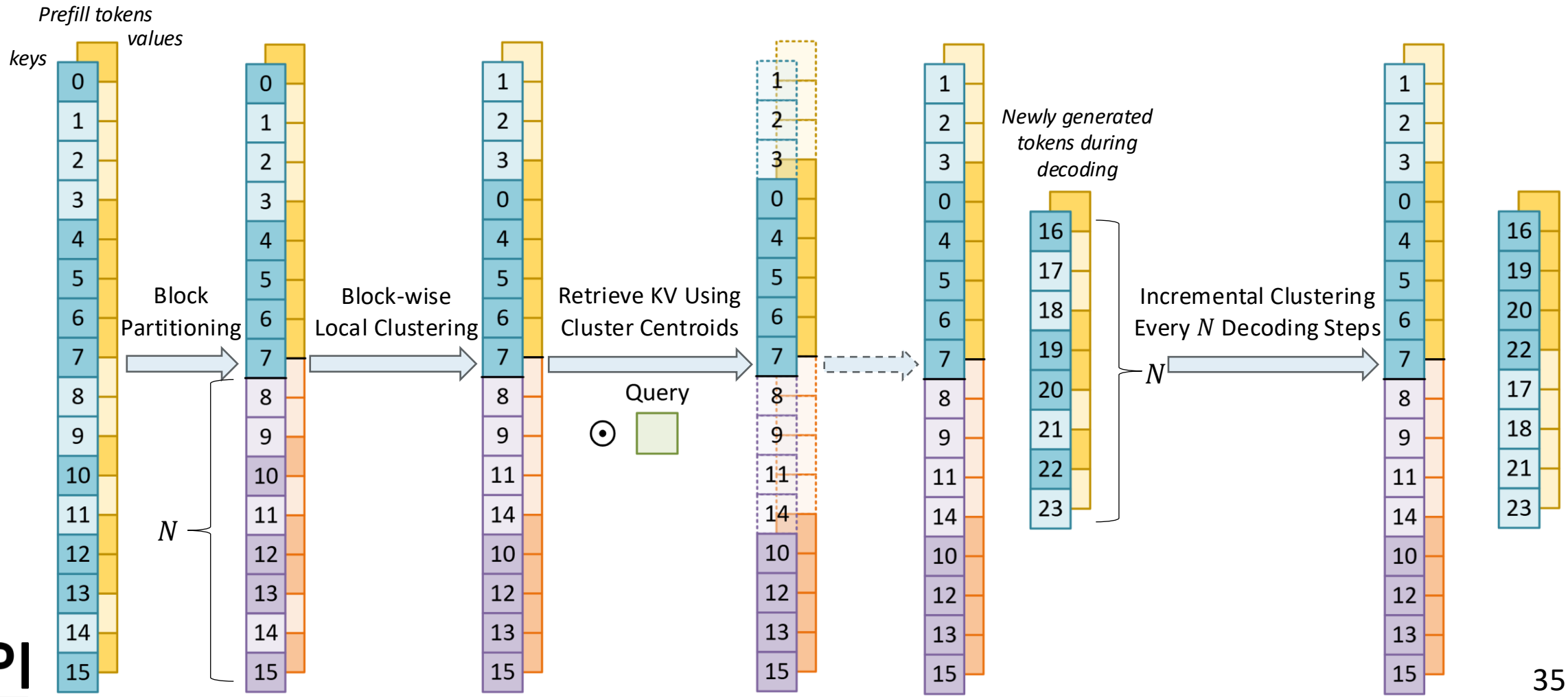
Incremental Semantic Clustering

- **Retrieval:** query • centroids → pick top clusters



Incremental Semantic Clustering

- **Decoding:** cluster every N new tokens → append clusters



Evaluation Methodology

- **System (Simulation):** 8 × NVIDIA H100 cores and 40 × HBM3 stacks (one GEMV unit per bank)
- **Benchmark:** LongBench, RULER
- **Workload:** LongChat-7B, LLaMA-3.1-8B, Mistral-7B, GPT-13B
- **Baseline Sparsity Methods:** Quest (page-wise), InfiniGen (token-wise), SparQ (token-wise)
- **KV Cache Budget:** default 1024

LongBench: Diverse Long-Context Downstream Tasks

- LongBench Results:

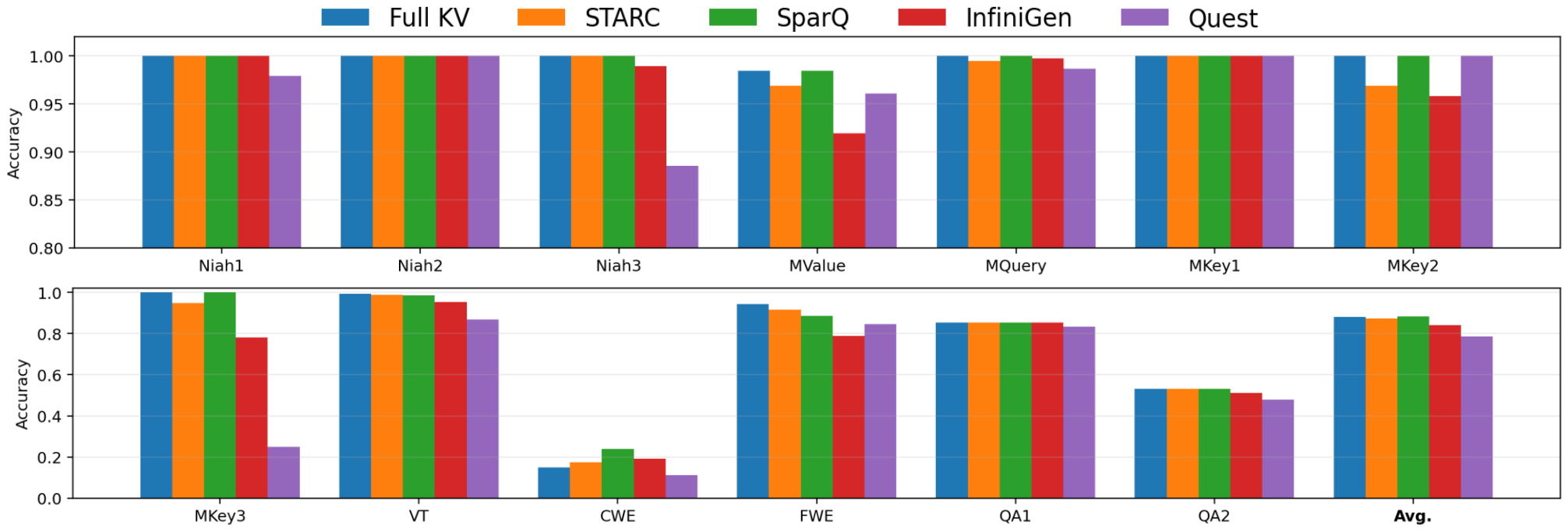
KV Budget: 1024	Single-Document QA			Multi-Document QA			Summarization			Few-Shot Learning			Synthetic		Code		
	NrtvQA	Qasper	MF-en	HotpotQA	2WikiMQA	Musique	GovReport	QMSum	MultiNews	TREC	TriviaQA	SAMSum	PCount	PRe	Lcc	RB-P	Avg.
LongChat																	
Full KV	19.51	25.98	**43.80**	31.94	**23.20**	11.38	**31.77**	21.66	26.06	**66.00**	82.00	20.79	**2.00**	30.00	53.86	48.68	**33.66**
STARC	17.55	29.44	40.92	32.32	19.29	9.73	31.22	22.08	25.01	64.00	80.80	21.82	**2.00**	32.00	**57.16**	48.82	**33.38**
SparQ	**19.56**	**29.90**	40.90	31.05	22.84	**12.92**	30.98	**23.19**	**26.49**	64.00	**84.53**	25.89	0.00	30.50	54.34	**55.72**	**34.55**
InfiniGen	15.41	29.56	41.92	**36.20**	20.35	8.89	29.36	22.22	24.73	64.00	84.38	**29.75**	**2.00**	32.00	51.84	51.06	**33.98**
Quest	14.58	29.23	43.67	28.37	18.62	10.51	29.12	22.29	24.91	**66.00**	79.31	20.88	**2.00**	**34.00**	52.60	49.00	**32.82**
Mistral																	
Full KV	23.94	40.07	57.58	49.10	36.71	22.27	35.66	**25.77**	26.80	**80.00**	87.67	47.35	4.00	98.00	58.98	56.36	**46.89**
STARC	19.97	34.93	57.70	**51.49**	35.48	23.39	**35.67**	24.72	26.72	76.00	**88.87**	**48.16**	2.00	98.00	**61.74**	55.76	**46.29**
SparQ	**29.36**	**40.93**	53.68	51.33	37.36	**27.22**	34.49	25.67	**27.66**	74.00	88.86	47.17	5.00	**99.00**	60.43	**62.14**	**47.77**
InfiniGen	23.34	37.73	**57.90**	51.41	**39.45**	19.69	35.06	24.89	26.29	76.00	85.67	47.60	2.00	98.00	59.82	59.58	**46.53**
Quest	22.79	30.88	52.39	47.12	38.63	18.73	33.45	24.23	27.26	66.00	88.42	44.73	**8.18**	92.00	60.86	57.52	**44.57**
Llama-3.1																	
Full KV	27.02	13.98	28.04	18.30	17.45	**13.01**	**35.83**	23.66	25.91	**74.00**	89.77	**44.56**	3.92	97.50	63.30	55.06	**39.46**
STARC	**31.73**	13.57	**28.14**	20.40	**18.08**	11.54	35.26	23.53	25.62	72.00	88.57	44.25	5.67	**98.33**	**64.30**	54.42	**39.71**
SparQ	29.53	13.83	26.97	17.64	16.85	10.27	33.95	**23.79**	**26.73**	71.00	**91.47**	44.20	**7.12**	98.21	64.19	**60.44**	**39.76**
InfiniGen	28.80	**14.15**	27.88	**24.27**	17.79	9.75	34.15	23.31	26.59	70.00	89.81	44.05	4.67	96.00	61.98	59.02	**39.51**
Quest	18.66	11.75	22.96	16.90	13.52	5.46	34.22	22.12	25.87	70.00	85.60	42.94	0.80	96.27	58.90	56.08	**36.38**

- **More effective** than page-wise (Quest)
- **Comparable** with toke-level (SparQ / InfiniGen)
- Especially strong on **GQA models** like LLaMA-3.1 and Mistral

RPI

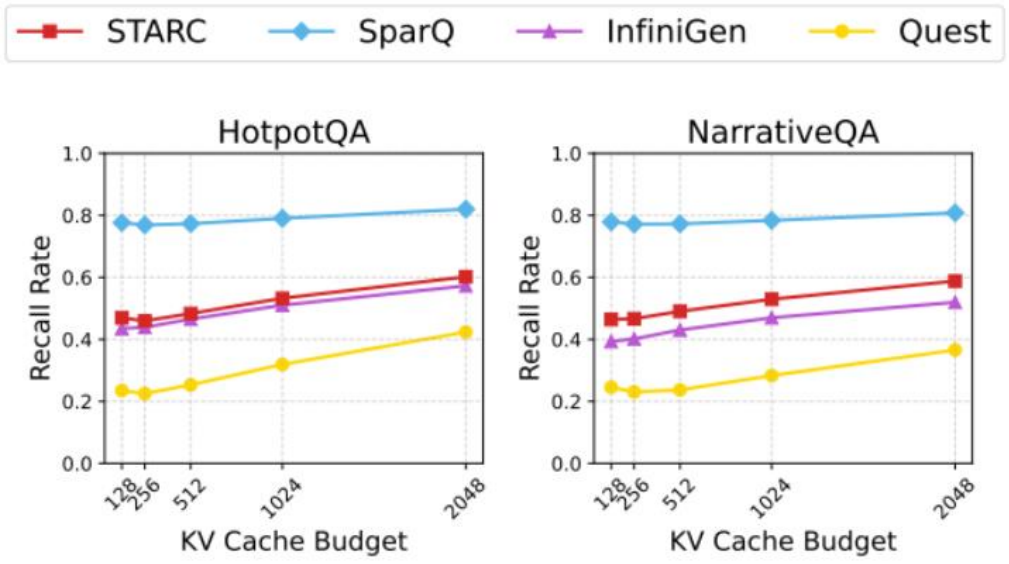
RULER: Ultra-Long-Context Robustness

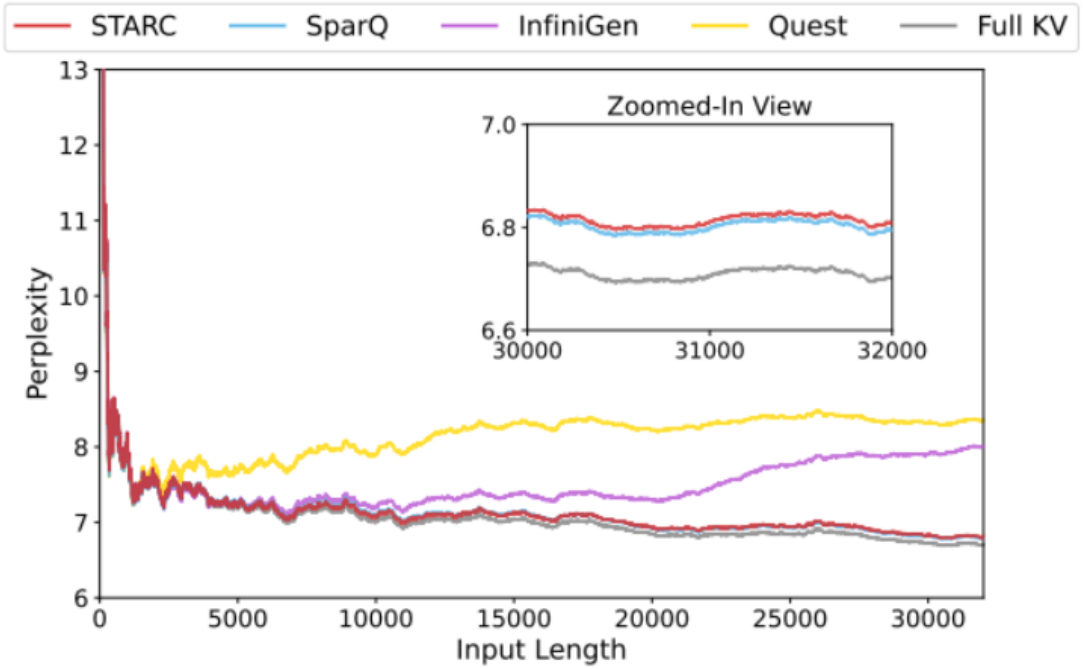
- RULER Result
  - **32K** context, KV budget = 1024
  - Avg. accuracy stays **close to** Full KV / SparQ, **better than** InfiniGen and Quest
  - Shows **robustness under ultra-long context**



Important-Token Recall & Language Modeling

- **Recall Rate:** Recall of top-attended tokens from full attention → **This explains why accuracy holds**
- Perplexity on PG-19 dataset evaluates long-context language modeling quality
  - STARC remains **very close to** strong baselines





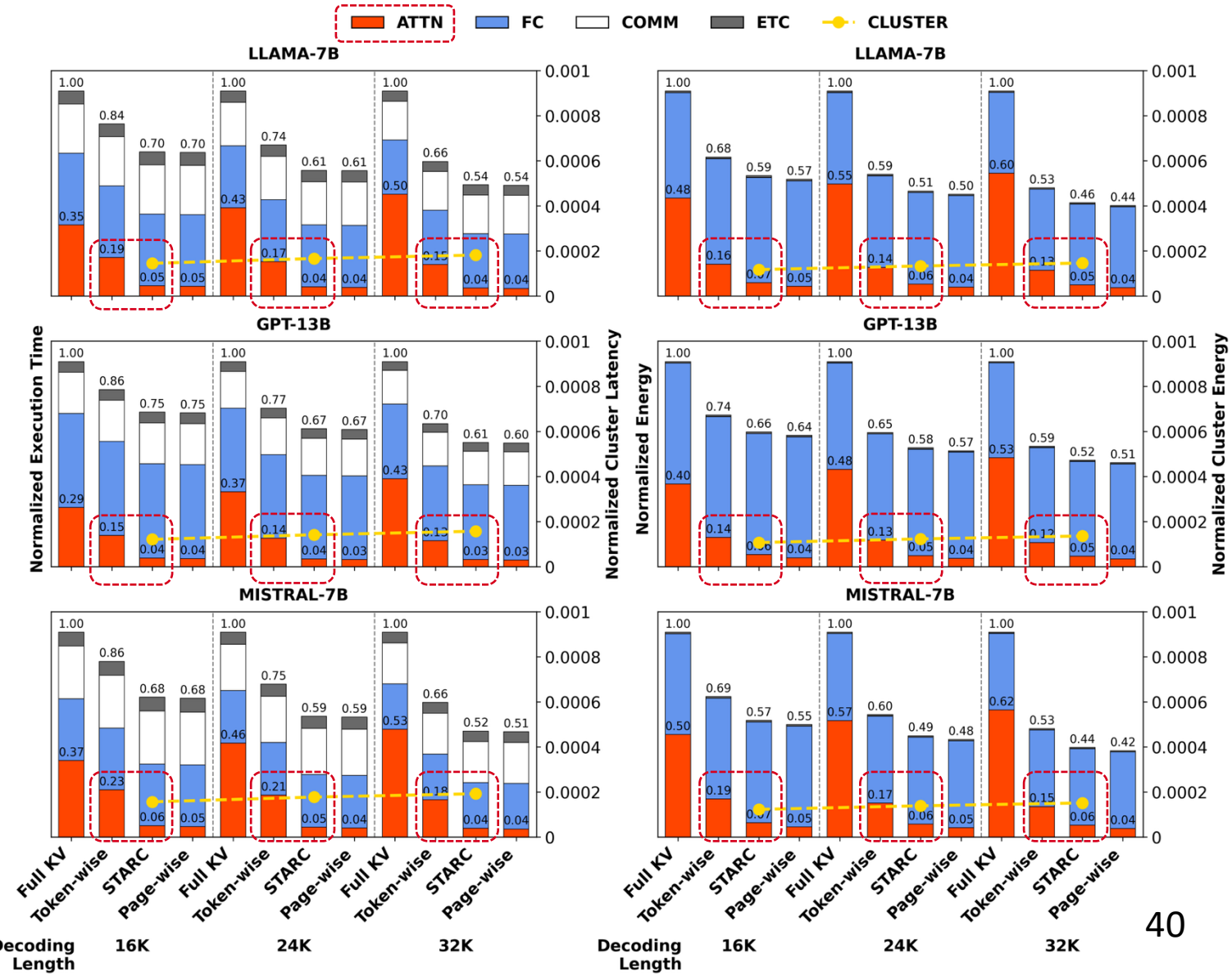
RPI

STARC Converts Sparsity into Latency and Energy Gains

- For Attention Layer:

Up to **78% latency** reduction vs. **token-wise sparsity**

Up to **65% energy** reduction vs. **token-wise sparsity**



STARC Converts Sparsity into Latency and Energy Gains

- For Attention Layer:

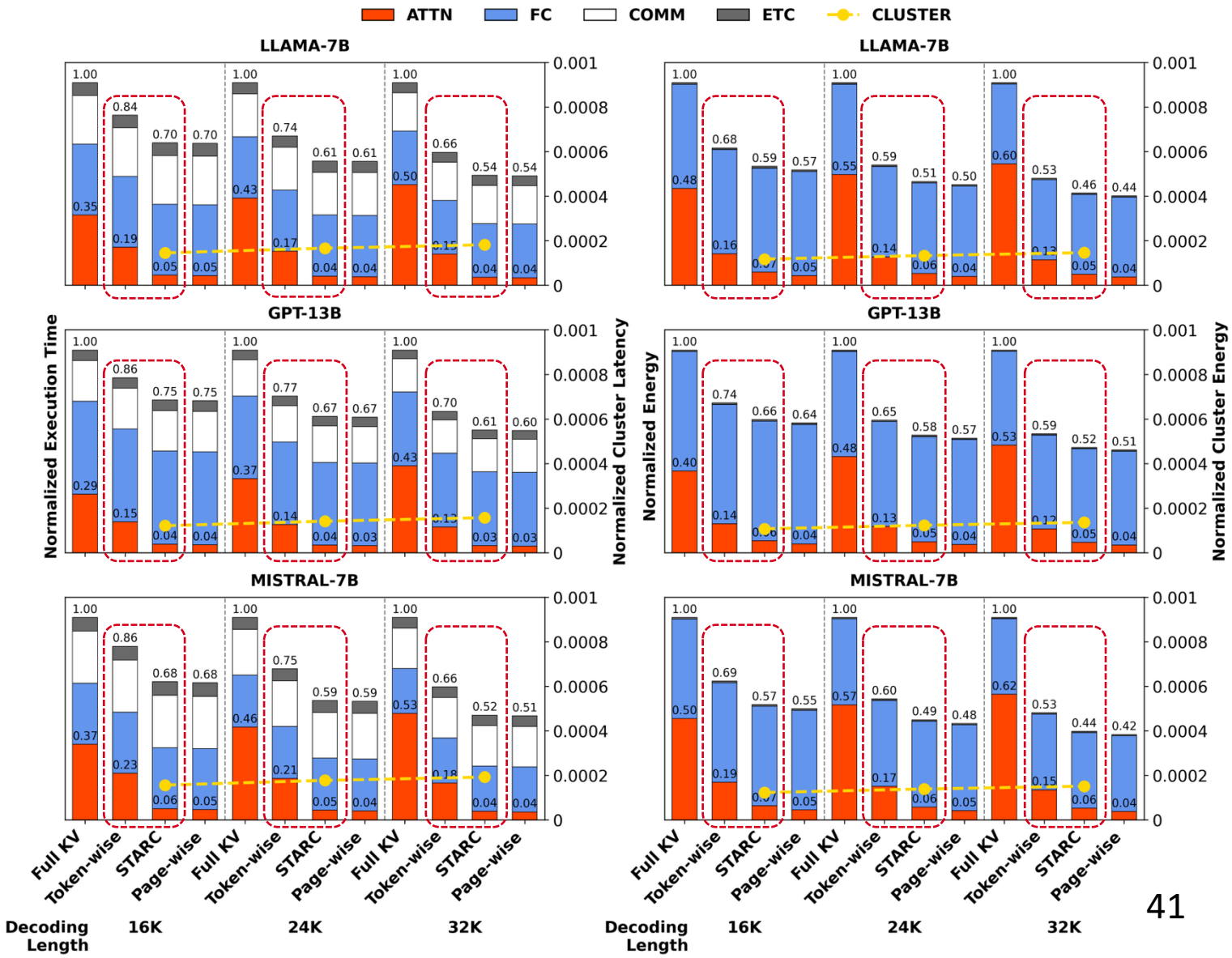
Up to **78% latency** reduction vs. token-wise sparsity

Up to **65% energy** reduction vs. token-wise sparsity

- For End-to-End Decoding:

13%–21% faster execution vs. token-wise sparsity

11%–18% lower energy vs. token-wise sparsity



STARC Converts Sparsity into Latency and Energy Gains

- **For Attention Layer:**

Up to **78% latency** reduction vs. **token-wise sparsity**

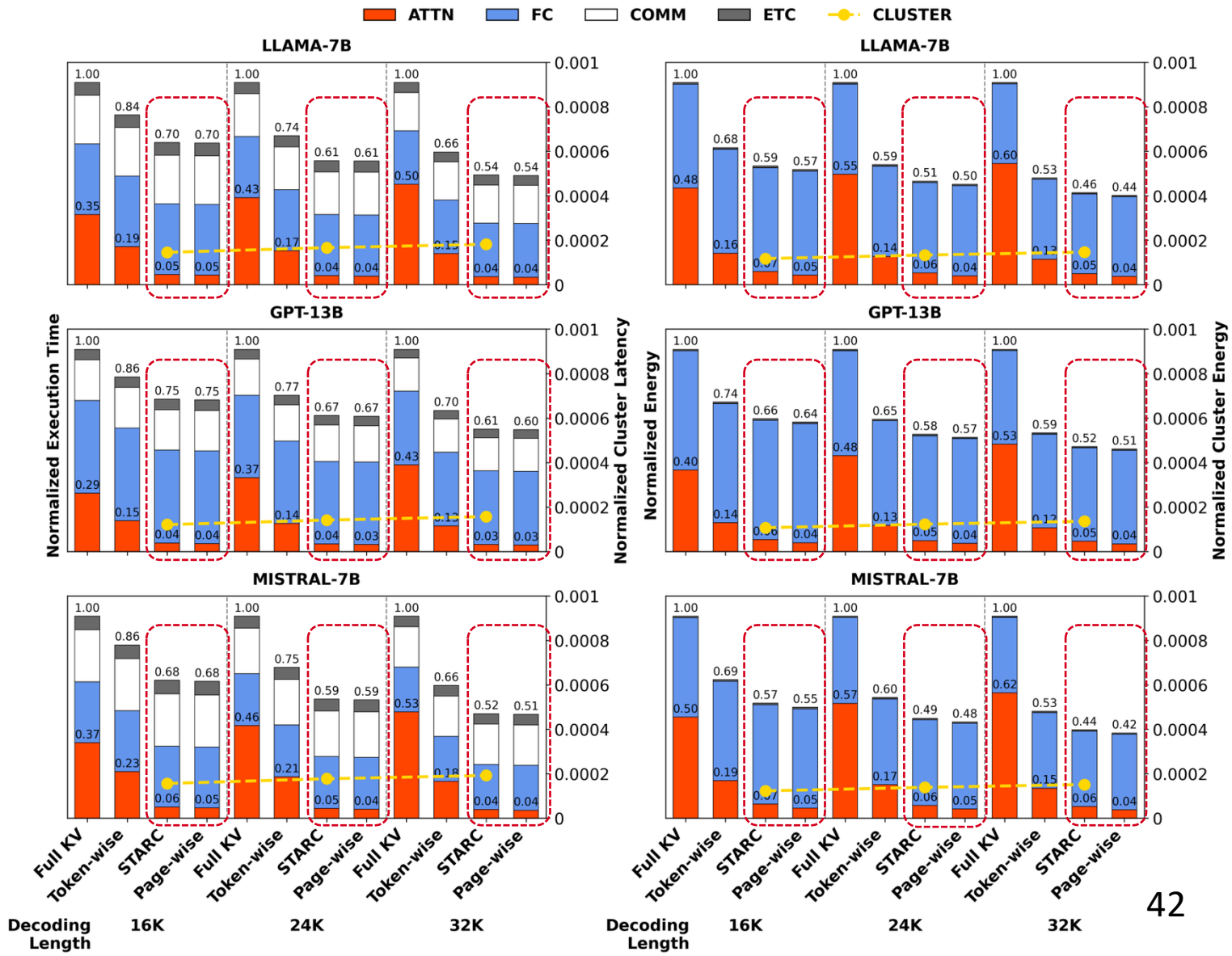
Up to **65% energy** reduction vs. **token-wise sparsity**

- **For End-to-End Decoding:**

13%–21% faster execution vs. **token-wise sparsity**

11%–18% lower energy vs. **token-wise sparsity**

- **STARC** is **close to** the idealized no-overfetch **page-wise** upper bound



STARC Converts Sparsity into Latency and Energy Gains

- For Attention Layer:

Up to **78% latency** reduction vs. **token-wise sparsity**

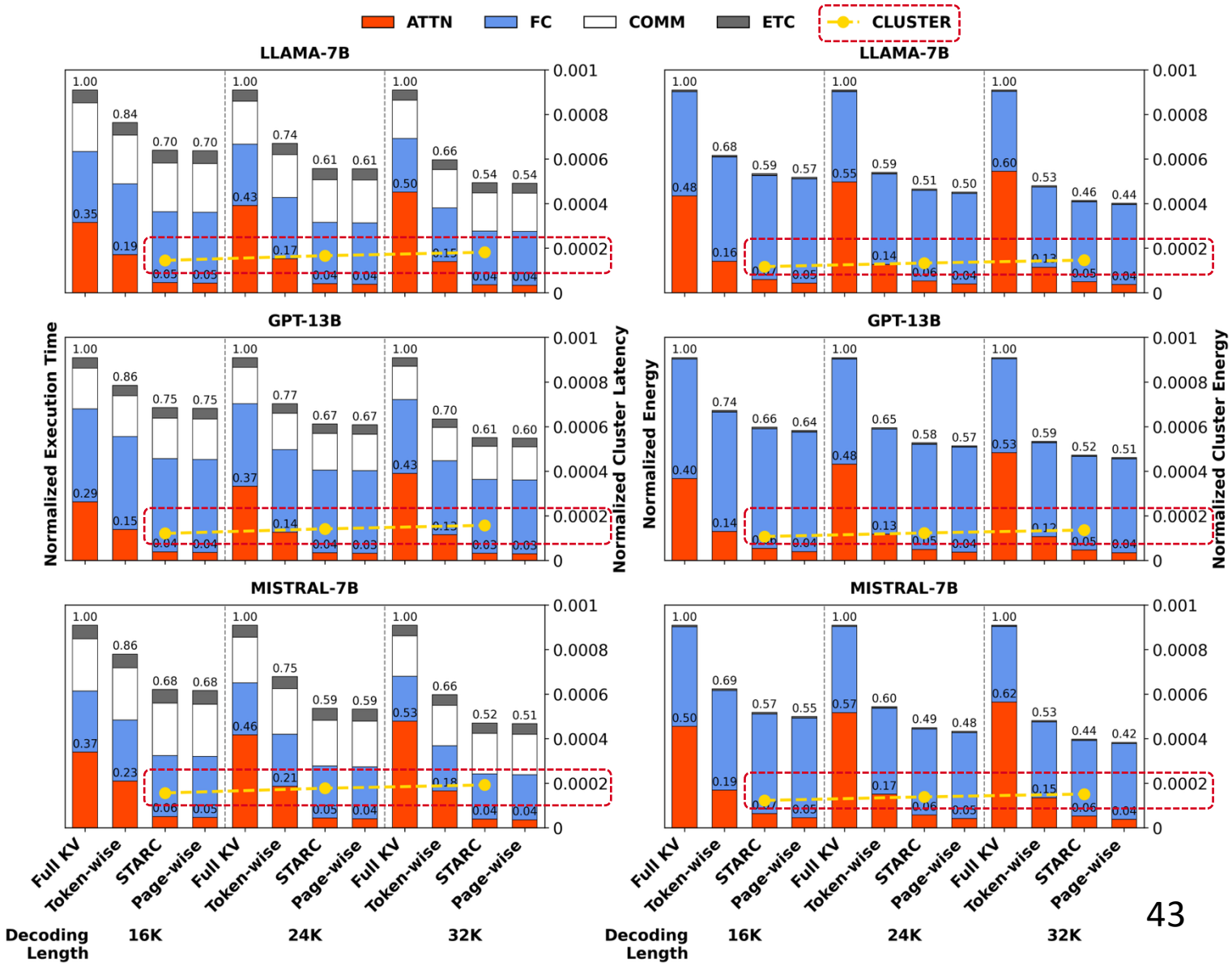
Up to **65% energy** reduction vs. **token-wise sparsity**

- For End-to-End Decoding:

13%–21% faster execution vs. **token-wise sparsity**

11%–18% lower energy vs. **token-wise sparsity**

- **STARC** is **close to** the idealized no-overfetch **page-wise** upper bound
- **Clustering overhead** ≈ **0.02%**



STARC: Selective Token Access with Remapping and Clustering for Efficient LLM Decoding on PIM Systems

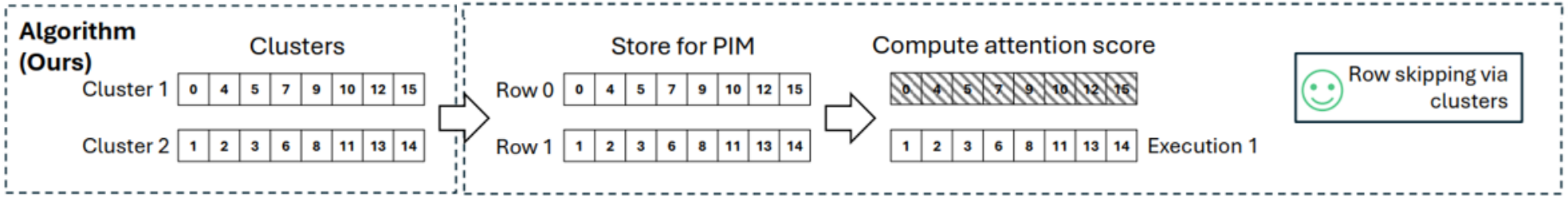
Problem:

- PIM executes attention **at row granularity**, while dynamic sparsity selects **scattered tokens**.
- This **mismatch** limits the practical benefit of sparse decoding on PIM.

Our Work:

- A **clustering-based** KV data mapping scheme for PIM-based LLM decoding.
- Group semantically similar KV pairs and store them contiguously.
- Enable **efficient sparse attention on PIM** with **strong accuracy** and **low overhead**.

Solution



RPI

Concluding Remarks

- From compute-centric to memory-centric
- Increasing need to manage contextual data
- Contextual sparsity:
  - Precision-scaling & selective token access
  - Memory architectural support
  - Model adaptation
- A new avenue for scaling LLM decoding

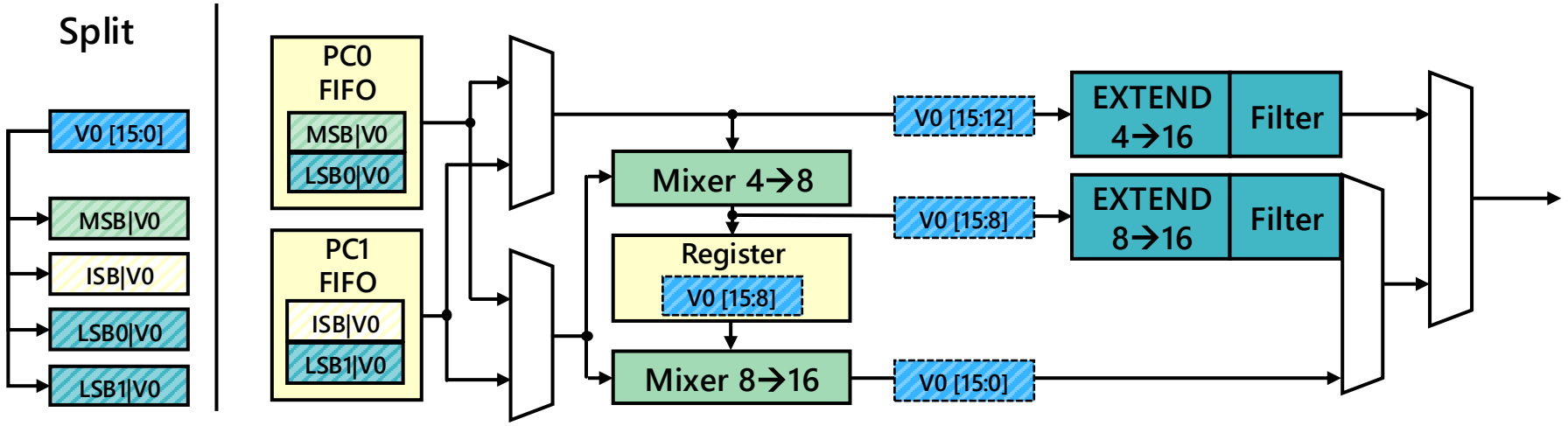
RPI

Backup Slides

RPI

Reconstruction Hardware

- Designed a low-complexity combiner unit
  - Unpacks received data
  - Uses minimal logic
  - Operates in 1-2 stages, depending on the precision



Synthesis & I/O Simulation

- Synthesized the unit at 45nm
  - Power assuming highest switching activity on all inputs
  - Low area & power despite large feature size
- Confirmed I/O performance on DRAMSim3
  - Consistent power draw
  - Consistent bandwidth

Parameter	Value	Unit
Area	20950	μm^2
Dynamic Power	4.48	mw
Leakage	63.29	μw

Synthesis Results

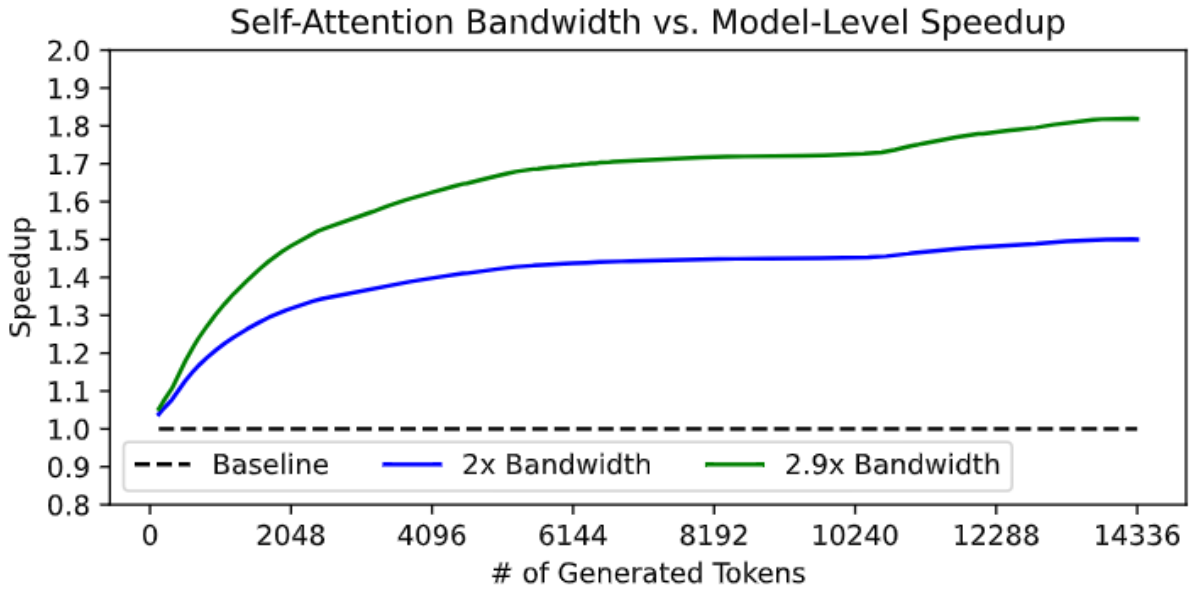
	Power (mW)			Bandwidth (GB/s)		
# Vals	FP16	FP8	FP4	FP16	FP8	FP4
16k	454.3	454.2	455.9	29.8	29.6	29.5
8k	454.3	454.0	442.8	29.7	29.6	29.5
4k	454.0	440.8	442.4	29.6	29.6	29.5
2k	441.0	440.4	442.0	29.7	29.6	29.5

DRAMSim3 Results

RPI

Model-Level Speedup

- Simulated the model speedup from higher bandwidth
  - A100 running LLaMa 3 8B
    - 2 evaluated bandwidths: 2x & 2.9x
  - High quantities of generated tokens benefit most from increased bandwidth



RPI

STARC Performs KV Clustering Directly in HBM-PIM

How?

- Map **cosine K-means** to existing PIM commands:
  - **Normalization:** MAC_AB + MVSB + VNORM
  - **Assignment:** WRGB + MAC_AB + MVSB
  - **Update:** MVGB + MAC_AB + WRGB

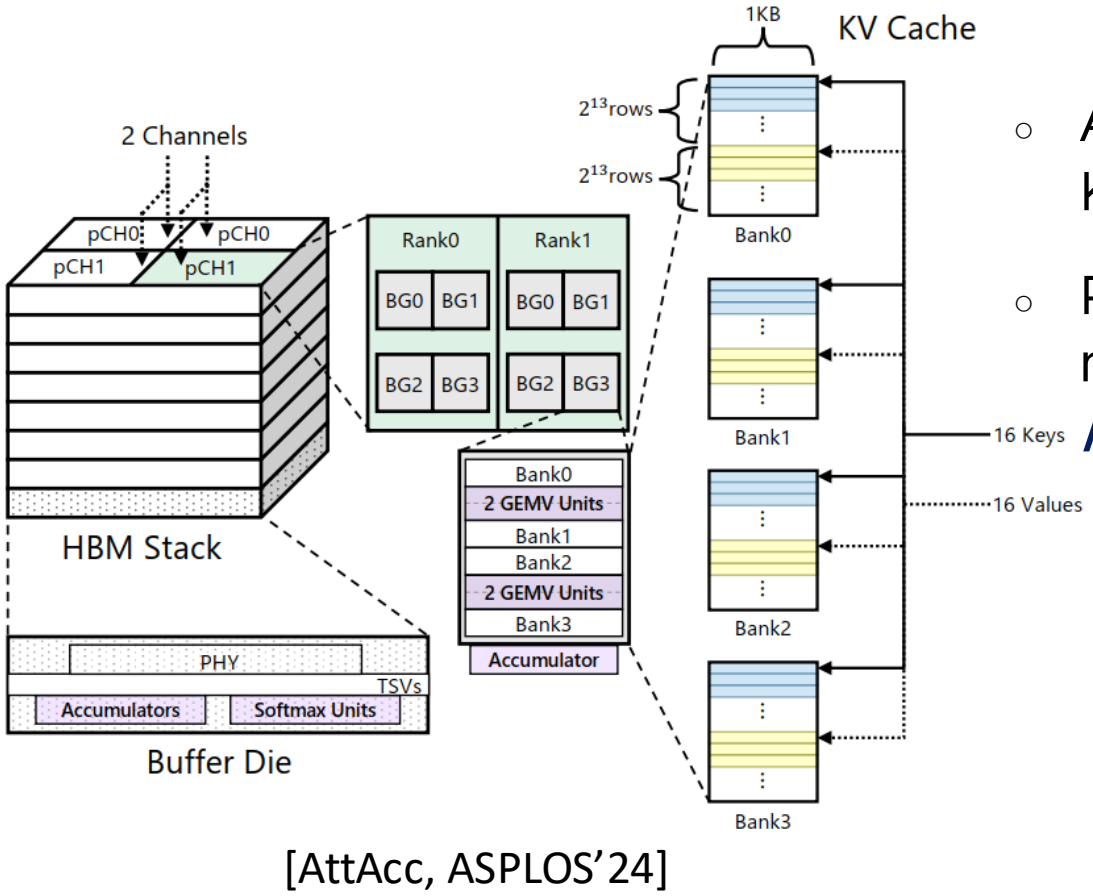
MAC_AB	dot product / accumulation
MVSB	move scores/norms to softmax buffer
VNORM (added)	computes $1/\sqrt{\|\|v\|\|^2}$ for cosine similarity
WRGB	write vectors / centroids
MVGB	broadcast vectors

Why low overhead?

- Reuses existing near-bank compute primitives
- No new hardware structures or additional area overhead

RPI

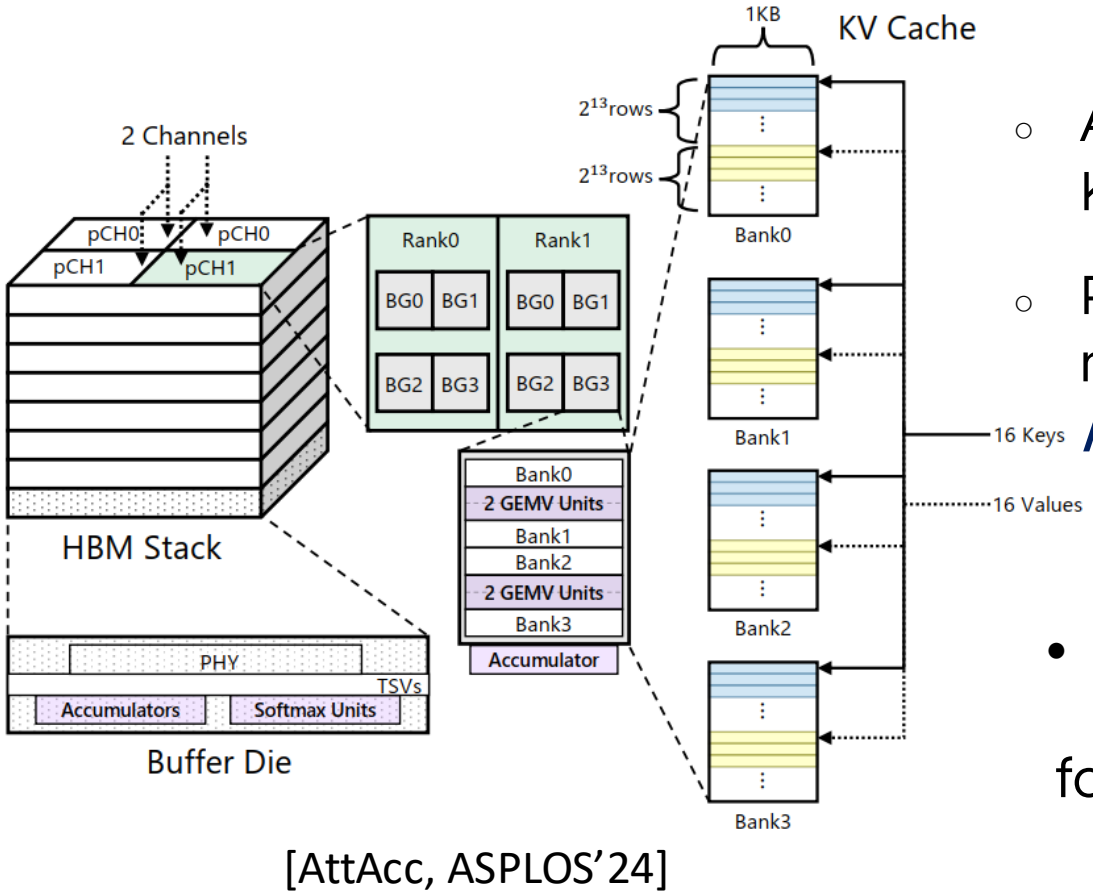
PIM-Aware Hardware/Algorithm Co-Design



[AttAcc, ASPLOS'24]

- Access Granularity: One row activation accesses 16 K/V vectors.
- Peak compute / internal bandwidth defines the memory-bound–to–compute-bound transition point: **/* ≈ 4 FLOPs/Byte**

PIM-Aware Hardware/Algorithm Co-Design



[AttAcc, ASPLOS'24]

- Access Granularity: One row activation accesses 16 K/V vectors.
- Peak compute / internal bandwidth defines the memory-bound–to–compute-bound transition point: **/* ≈ 4 FLOPs/Byte**

- On the algorithm side:
  K-means arithmetic intensity satisfies $AI \approx K$ for $N \gg K$.
  Therefore, choose $\boldsymbol{K = /* = 4}$ to balance compute and memory bandwidth utilization.

RPI

References

1. Jaehyun Park, Jaewan Choi, Kwanhee Kyung, Michael Jaemin Kim, Yongsuk Kwon, Nam Sung Kim, and Jung Ho Ahn. 2024. AttAcc! Unleashing the power of PIM for batched transformer-based generative model inference. In Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2. 103–119.
2. Guseul Heo, Sangyeop Lee, Jaehong Cho, Hyunmin Choi, Sanghyeon Lee, Hyungkyu Ham, Gwangsun Kim, Divya Mahajan, and Jongse Park. 2024. Neupims: Npu-pim heterogeneous acceleration for batched llm inferencing. In Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3. 722–737.
3. Minxuan Zhou, Weihong Xu, Jaeyoung Kang, and Tajana Rosing. 2022. TransPIM: A memory-based acceleration via software-hardware codesign for transformer. In 2022 IEEE International Symposium on High Performance Computer Architecture (HPCA). IEEE, 1071–1085.
4. Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. 2024. Quest: Query-aware sparsity for efficient long context llm inference. arXiv preprint arXiv:2406.10774 (2024).
5. Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. InfiniGen: Efficient generative inference of large language models with dynamic KV cache management. In 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24). 155–172.
6. Luka Ribar, Ivan Chelombiev, Luke Hudlass-Galley, Charlie Blake, Carlo Luschi, and Douglas Orr. 2023. Sparq attention: Bandwidth-efficient LLM inference. arXiv preprint arXiv:2312.04985 (2023).
7. Lian Liu, Shixin Zhao, Bing Li, Haimeng Ren, Zhaohui Xu, Mengdi Wang, Xiaowei Li, Yinhe Han, and Ying Wang. 2025. Make LLM Inference Affordable to Everyone: Augmenting GPU Memory with NDP-DIMM. In 2025 IEEE International Symposium on High Performance Computer Architecture (HPCA). IEEE, 1751–1765.
8. Guangda Liu, Chengwei Li, Jieru Zhao, Chenqi Zhang, and Minyi Guo. 2024. Clusterkv: Manipulating llm kv cache in semantic space for recallable compression. arXiv preprint arXiv:2412.03213 (2024).

RPI

STARC: Selective Token Access with Remapping and Clustering for Efficient LLM Decoding on PIM Systems

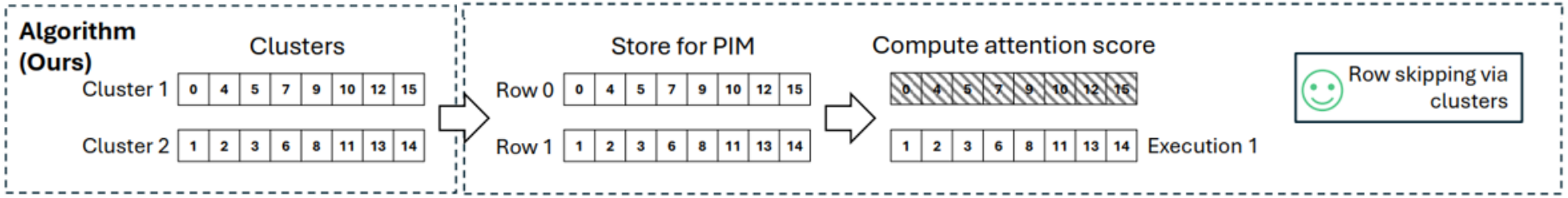
Problem:

- PIM executes attention **at row granularity**, while dynamic sparsity selects **scattered tokens**.
- This **mismatch** limits the practical benefit of sparse decoding on PIM.

Our Work:

- A **clustering-based** KV data mapping scheme for PIM-based LLM decoding.
- Group semantically similar KV pairs and store them contiguously.
- Enable **efficient sparse attention on PIM** with **strong accuracy** and **low overhead**.

Solution



RPI

STARC Performs KV Clustering Directly in HBM-PIM

Operation	Command Count	MAC	Read Bytes	Write Bytes
Normalization (per vector)				
MAC_AB(self-dot)	T_D	D	DS	–
MVSB(norm)	1	0	–	S
VNORM(vector/$\sqrt{\cdot}$)	T_D	D	DS	–
Total / vector	–	$2D$	$2DS$	S
Assignment (per iteration)				
WRGB(samples)	N	0	–	NDS
MAC_AB	$NT_D \times T_K$	NKD	samples: NDS, centroids: KDS	–
MVSB(scores)	NT_K	0	–	NKS
Host(argmax)	–	–	NKS	only labels
Total / iteration	–	NKD	$(ND + KD + NK)S$	$NDS + NKS$
Update (per iteration)				
MVGB(broadcast v_i)	N	0	NDS	–
MAC_AB (accumulation & averaging)	NT_D	$(ND + KD)/2$	–	–
WRGB(new μ_k)	1	0	–	KDS
Total / iteration	–	$(ND + KD)/2$	NDS	KDS

RPI

LongBench Results

Table 4. LongBench results for STARC and baseline sparsity methods (KV cache budget: 256 tokens).

	Single-Document QA			Multi-Document QA			Summarization			Few-Shot Learning			Synthetic		Code		
	NrtvQA	Qasper	MF-en	HotpotQA	2WikiMQA	Musique	GovReport	QMSum	MultiNews	TREC	TriviaQA	SAMSum	PCount	PRe	Lcc	RB-P	Avg.
KV Budget: 256																	
LongChat																	
Full KV	19.51	25.98	**43.80**	31.94	23.20	11.38	**31.77**	21.66	26.06	**66.00**	82.00	20.79	**2.00**	30.00	53.86	48.68	33.66
STARC	18.82	28.35	34.79	**34.41**	18.64	8.10	30.50	21.74	24.64	62.00	81.01	24.17	**2.00**	**32.00**	55.56	45.00	32.61
SparQ	**19.87**	**30.77**	40.71	31.70	20.93	**12.89**	30.93	**22.80**	**26.38**	64.00	**85.17**	31.37	0.50	31.50	**55.63**	**55.58**	35.05
InfiniGen	13.68	27.47	36.05	27.86	20.41	7.75	26.27	20.49	24.97	62.00	77.22	**32.47**	**2.00**	18.00	52.70	50.28	31.23
Quest	10.49	26.47	34.90	20.04	**24.23**	12.53	21.59	20.48	25.29	56.00	63.80	22.62	**2.00**	28.00	47.86	38.58	28.43
Mistral																	
Full KV	23.94	40.07	**57.58**	49.10	36.71	22.27	**35.66**	25.77	26.80	**80.00**	87.67	47.35	4.00	98.00	58.98	56.36	46.89
STARC	20.19	35.71	56.50	44.43	**45.85**	20.32	34.06	24.06	26.54	68.00	87.11	**48.97**	**6.00**	88.00	**61.94**	57.60	45.33
SparQ	**27.12**	**40.87**	53.94	**49.32**	39.51	**23.97**	35.31	25.14	**27.48**	73.00	**88.78**	47.28	4.50	**99.50**	61.56	**63.15**	47.53
InfiniGen	19.52	37.95	54.54	42.28	38.98	10.34	31.14	22.82	27.21	74.00	83.45	47.70	4.00	90.00	61.70	52.34	43.62
Quest	16.81	30.88	36.99	35.62	27.66	10.12	29.18	21.11	26.04	66.00	78.39	37.84	4.89	83.50	57.22	43.98	37.89
Llama-3.1																	
Full KV	27.02	13.98	28.04	18.30	17.45	13.01	**35.83**	**23.66**	25.91	**74.00**	89.77	44.56	3.92	**97.50**	63.30	55.06	39.46
STARC	**30.84**	12.91	26.42	**21.88**	**18.34**	**13.48**	34.96	22.18	25.46	66.00	86.71	**44.94**	**12.00**	94.33	65.52	57.32	39.58
SparQ	29.70	12.35	26.97	17.69	15.31	11.40	33.89	23.38	**27.00**	70.00	**92.19**	44.58	6.90	**97.50**	64.55	**60.79**	39.64
InfiniGen	21.86	**16.53**	**29.63**	21.47	17.76	5.36	32.38	22.70	25.50	68.00	86.40	44.58	7.25	96.00	**67.36**	55.38	38.64
Quest	8.68	9.90	18.18	12.19	9.48	3.02	25.33	18.36	23.50	44.00	73.23	31.53	3.55	83.00	51.90	46.52	28.90

RPI

LongBench Results

Table 5. LongBench results for STARC and baseline sparsity methods (KV cache budget: 512 and 2048 tokens).

	Single-Document QA			Multi-Document QA			Summarization			Few-Shot Learning			Synthetic		Code		
	NrtvQA	Qasper	MF-en	HotpotQA	2WikiMQA	Musique	GovReport	QMSum	MultiNews	TREC	TriviaQA	SAMSum	PCount	PRe	Lcc	RB-P	Avg.
KV Budget: 512																	
LongChat																	
Full KV	**19.51**	25.98	**43.80**	31.94	23.20	11.38	**31.77**	21.66	26.06	**66.00**	82.00	20.79	**2.00**	30.00	53.86	48.68	33.66
STARC	17.59	**29.86**	39.44	**33.92**	18.70	10.21	30.46	20.49	25.11	64.00	79.81	22.48	**2.00**	30.00	**57.60**	48.38	33.13
SparQ	19.20	29.18	40.81	32.27	22.30	13.43	30.81	**22.81**	**26.29**	64.50	**84.70**	29.05	0.00	30.00	55.11	**55.70**	34.76
InfiniGen	16.37	25.37	38.10	28.48	18.15	**13.52**	28.71	21.10	25.06	64.00	79.03	**31.93**	0.00	28.00	52.60	53.42	32.74
Quest	13.39	28.08	40.90	25.34	**24.59**	7.59	27.82	21.48	25.39	**66.00**	76.67	21.94	0.00	**36.00**	53.36	45.08	32.10
Mistral																	
Full KV	23.94	40.07	57.58	49.10	36.71	22.27	**35.66**	**25.77**	26.80	**80.00**	87.67	47.35	4.00	98.00	58.98	56.36	46.89
STARC	21.49	37.26	**58.73**	47.18	**40.15**	23.68	34.32	23.66	26.64	74.00	87.67	48.38	4.00	94.00	62.18	58.90	46.39
SparQ	**29.00**	**40.09**	53.70	**50.43**	37.75	**26.49**	34.23	25.68	**27.50**	74.00	**89.07**	47.35	5.00	**99.50**	60.72	**62.07**	47.66
InfiniGen	22.76	36.82	58.67	49.17	31.64	15.34	33.80	23.87	26.51	78.00	83.67	**49.67**	2.00	96.00	**62.88**	58.04	45.55
Quest	18.39	33.14	45.93	41.79	33.64	18.21	32.57	22.77	26.45	64.00	84.50	41.63	**6.50**	92.67	59.92	49.84	42.00
Llama-3.1																	
Full KV	27.02	13.98	28.04	18.30	17.45	13.01	**35.83**	23.66	25.91	**74.00**	89.77	44.56	3.92	97.50	63.30	55.06	39.46
STARC	**31.78**	13.06	**28.77**	18.49	**18.58**	**14.24**	34.33	22.65	25.80	70.00	88.57	44.26	4.67	95.83	63.56	60.08	39.67
SparQ	30.30	13.30	26.19	17.90	16.12	10.43	34.11	**23.83**	**27.17**	70.50	**91.97**	43.80	**8.29**	**98.08**	64.43	**61.34**	39.86
InfiniGen	23.36	**16.90**	27.18	**22.17**	18.30	8.76	33.69	22.79	25.85	66.00	89.90	**45.10**	6.67	98.07	**66.46**	50.72	38.87
Quest	15.57	10.77	21.82	12.42	13.25	5.93	29.48	22.05	26.65	60.00	78.87	37.44	2.54	90.52	61.18	53.94	33.90

RPI

LongBench Results

KV Budget: 2048

LongChat																
Full KV	19.51	25.98	43.80	31.94	23.20	11.38	**31.77**	21.66	26.06	**66.00**	82.00	20.79	**2.00**	30.00	53.86	48.68	33.66
STARC	17.70	28.27	41.15	**33.75**	23.28	11.21	30.97	**23.13**	26.50	63.00	81.53	20.80	1.00	31.00	54.15	50.92	33.65
SparQ	**20.01**	28.48	42.21	31.02	**23.53**	12.68	31.06	23.07	**26.69**	65.00	**84.41**	24.61	0.00	30.00	52.86	**55.69**	34.46
InfiniGen	15.76	30.35	40.52	31.81	20.05	9.00	29.82	22.13	26.00	62.00	81.78	**25.94**	0.00	**36.00**	**57.62**	50.38	33.70
Quest	14.93	**31.48**	**45.33**	31.60	19.70	**12.93**	30.83	22.07	25.61	62.00	81.33	20.35	**2.00**	30.00	55.68	49.92	33.49
Mistral																
Full KV	23.94	40.07	57.58	49.10	36.71	22.27	**35.66**	**25.77**	26.80	**80.00**	87.67	47.35	4.00	98.00	58.98	56.36	46.89
STARC	28.71	**43.73**	54.06	48.62	37.87	23.36	34.82	25.75	**27.87**	72.00	85.76	**47.87**	**9.00**	**100.00**	59.64	57.91	47.31
SparQ	**29.58**	40.25	53.37	**51.01**	37.94	**27.22**	34.45	25.68	27.76	74.50	**89.06**	47.01	5.00	99.00	**59.76**	**62.04**	47.73
InfiniGen	25.34	39.30	**59.51**	50.20	**41.79**	18.54	34.83	24.68	26.85	78.00	87.67	47.38	2.00	96.00	58.96	59.46	46.91
Quest	23.48	40.55	58.73	48.94	37.63	25.41	32.79	24.07	27.28	70.00	88.33	47.07	6.00	98.00	57.86	60.54	46.67
Llama-3.1																
Full KV	27.02	**13.98**	28.04	18.30	17.45	**13.01**	**35.83**	**23.66**	25.91	**74.00**	89.77	44.56	3.92	97.50	63.30	55.06	39.46
STARC	**30.61**	13.88	27.94	**20.85**	**19.62**	11.53	34.56	22.75	26.30	72.00	88.57	**45.54**	2.92	**99.00**	62.66	55.54	39.64
SparQ	29.76	13.06	26.61	17.30	16.85	11.26	34.02	23.50	**26.69**	71.00	91.48	43.82	6.37	98.01	**63.38**	**59.78**	39.56
InfiniGen	27.98	13.33	**32.01**	19.49	18.79	12.86	35.45	23.10	26.66	72.00	89.81	44.63	**7.00**	96.67	61.12	55.74	39.79
Quest	24.41	13.34	23.39	15.97	15.59	10.59	35.03	23.33	25.58	**74.00**	**92.60**	45.23	5.18	97.50	59.44	56.52	38.61

RPI